

分类号: _____

单位代码: 10335

密 级: 公开

学 号: 22141095

浙江大学

硕士专业学位论文



中文论文题目: 逻辑运算可配置的高能效 SRAM 全数字存算
一体芯片设计研究

英文论文题目: A SRAM based Compute-In-Memory Macro
with Reconfigurable Bitwise Operation For AI

申请人姓名: 孟依绮

指导教师: 朱晓雷

合作导师: _____

专业名称: 集成电路工程

研究方向: 存内计算; 模拟集成电路设计

所在学院: 集成电路学院

论文提交日期: 2024.3.13

逻辑运算可配置的高能效 SRAM 全数字存算

一体芯片设计研究



论文作者签名: _____

指导教师签名: _____

论文评阅人 1: _____

评阅人 2: _____

评阅人 3: _____

评阅人 4: _____

评阅人 5: _____

答辩委员会主席: 虞小鹏/教授

委员 1: 黄科杰/研究员

委员 2: 葛海通/高级工程师

委员 3: _____

委员 4: _____

委员 5: _____

答辩日期: 2024.3.12

摘要

随着计算机技术的快速发展,内存速度的缓慢增长导致了“存储墙”问题的出现。存算一体架构作为解决这一问题的新兴技术,通过将存储和计算整合在一起,减少了处理器与存储器之间的数据传输频率,从而提高了能效。在存算技术领域,模拟存算通常能够更快地处理数据并且更加节能,但它们容易受到噪声的干扰。与此相反,数字存算则提供了更高精度和稳定性的数据。尽管如此,目前大多数数字存算解决方案仅支持单一的逻辑运算类型,使其应用受限,并且由于其庞大的运算电路,导致功耗较高。本文提出了一种逻辑运算可配置的高能效 SRAM 全数字存算一体架构及电路。

本文首先从模拟和数字角度对现有文献进行了梳理和分析,详细阐述了基于 SRAM 的存算电路的工作原理和设计要点,并简要概述了存内计算技术的主要应用场景。随后,针对 SRAM 存算一体芯片的功能扩展和高能效的要求,提出并设计了 SRAM 存算架构中的关键模块,逻辑运算可配置 SRAM 存算单元电路和低功耗、低成本加法器树电路。本文的创新点是:一种可配置的存算单元,由 4T (Transistor) 运算电路与 6T SRAM 电路组合,配合输入驱动模块可实现多种运算逻辑的切换,包括乘加运算和布尔逻辑运算(如 OR、AND、XOR);一种加法器树电路,采用精简的 16T 加法器,配合少量级间驱动电路,可在满足速度要求的同时,能够实现最低的功耗。

基于以上设计基础工作,本文使用 28nm 标准 CMOS 工艺,设计了一个 64Kb 容量的逻辑运算可配置的 SRAM 全数字存内计算宏电路,版图面积为 0.5mm^2 ,后端仿真结果表明,在输入与权重均为 4 比特的情况下,本文提出架构的吞吐率可达到 3300TOPS,能效比可达到 83.3TOP/W。相比于现有工作,在吞吐率和能效比方面得到了有效提升,同时还能完成多种运算,比如卷积运算和汉明距离的计算,提高了存算架构在不同应用场景中的灵活性和适用性。

关键词: 存算一体; SRAM; 加法树; 可配置的逻辑运算; 神经网络

ABSTRACT

With the rapid advancement of computer technology, the slow growth of memory speed has led to the emergence of the "memory wall" problem. The computing-in-memory (CIM) architecture proposed in this paper, as an emerging solution, integrates storage and computation, reducing the data transfer frequency between processor and memory, thus enhancing energy efficiency. In the field of CIM technology, analog CIM can process data faster and more energy-efficiently but is susceptible to noise. In contrast, digital CIM offers higher precision and stability. However, most digital CIM solutions currently support only single logic operations and have high power consumption due to their large operational circuits. This paper proposes a high-efficiency SRAM-based digital CIM architecture with configurable logic operations.

The paper begins with an analysis of existing literature from analog and digital perspectives, explaining the principles and design points of SRAM-based CIM circuits, and briefly outlines the main applications of in-memory computing technology. Subsequently, to meet the requirements of functional expansion and high efficiency of SRAM CIM chips, key modules are proposed and designed in the SRAM CIM architecture, including a logic-configurable SRAM CIM unit circuit and a low-power, low-cost adder tree circuit. The configurable CIM unit combines a 4T(Transistor) operational circuit with a 6T SRAM circuit, and in conjunction with an input driving module, can switch between various operational logics, including multiply-accumulate operations and Boolean logic operations (such as OR, AND, XOR). The adder tree circuit uses a streamlined 16T adder, combined with a few levels of driving circuits, to achieve the lowest power consumption while meeting speed requirements.

Based on the above design, this paper uses a 28nm standard CMOS process to design a 64Kb capacity logic-configurable SRAM full digital CIM macro circuit with a layout area of 0.5mm². Backend simulation results show that when the input and weight are both 4 bits, the throughput of the architecture proposed in this paper can reach 3300TOPS, and the energy efficiency can reach 83.3TOP/W. Compared with existing work, it has effectively improved throughput and energy efficiency, and can also complete various operations, such as convolution operations and Hamming distance calculations, improving the flexibility and applicability of the storage and computing architecture in different application scenarios.

KEY WORDS: Computing-in-Memory; SRAM; Adder Tree; Configurable Logic Operation; Neural Network

目 录

第 1 章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 基于非 SRAM 的存内计算研究现状	3
1.2.2 基于 SRAM 的存内计算研究现状	3
1.3 论文的研究内容与创新点	5
1.4 论文结构.....	6
第 2 章 基于 SRAM 的存算一体电路及工作原理.....	8
2.1 基于 SRAM 的存储架构	8
2.1.1 SRAM 存储阵列	8
2.1.2 SRAM 存储阵列的外围电路	11
2.2 基于 SRAM 的存算一体技术简介	13
2.2.1 存算一体架构分类.....	13
2.2.2 卷积操作概述.....	14
2.2.3 汉明距离的运算.....	16
2.3 小结.....	17
第 3 章 存算一体技术中关键模块的设计	18
3.1 面向卷积神经网络的存内计算系统模型	18
3.2 可配置逻辑运算的 SRAM 单元设计与分析	19
3.2.1 6T SRAM 单元的设计	19
3.2.2 可配置逻辑运算电路的设计.....	20
3.2.3 SRAM 存算单元的仿真与分析	22
3.3 加法器树方案比较.....	26
3.3.1 全加器电路.....	26
3.3.2 28T 与 16T 交替的加法器树.....	29
3.3.3 全 16T 的加法器树	31
3.3.4 混合华莱士树结构的加法器树.....	32
3.3.5 加法器树的仿真分析.....	34
3.4 小结.....	44
第 4 章 基于 SRAM 的可配置逻辑的存算一体架构及电路设计	45
4.1 可配置逻辑的存算一体技术基本架构	45
4.2 SRAM-CIM 阵列设计	46
4.2.1 输入控制模块.....	47
4.2.2 译码器电路设计.....	48

4.2.3 灵敏放大器设计	48
4.2.4 读写自适应模块设计	49
4.3 加法器树结构设计	50
4.4 移位累加模块设计	50
4.5 可配置的逻辑运算应用方法	51
4.5.1 面向 CNN 的存内计算	51
4.5.2 面向汉明距离的存内计算	52
4.6 可配置逻辑运算的存算架构仿真分析	54
4.6.1 SRAM 阵列的读写速度及功耗	54
4.6.2 累加器仿真	55
4.6.3 基于 SRAM 的可配置逻辑运算的存算一体架构仿真	56
4.7 小结	62
第 5 章 结论与展望	63
5.1 论文工作总结	63
5.2 研究方向展望	64
参考文献	65

图目录

图 1-1 传统架构的存储瓶颈.....	1
图 1-2 存算架构示意图.....	2
图 2-1 SRAM 阵列电路图	8
图 2-2 6T SRAM 单元电路图及写入波形	9
图 2-3 6T SRAM 单元读操作及其波形图	10
图 2-4 SRAM 的蝴蝶曲线	10
图 2-5 预充电电路及时序	11
图 2-6 self-timing 电路结构图	12
图 2-7 2-4 译码器与 3-8 译码器电路图	13
图 2-8 模拟存算与数字存算.....	14
图 2-9 卷积操作示意图.....	15
图 2-10 汉明距离计算示意图.....	16
图 3-1 面向 CNN 的基于 SRAM 的存算一体架构	19
图 3-2 4T 计算单元和 6T SRAM 电路图	19
图 3-3 可配置的逻辑运算单元电路图.....	21
图 3-4 使用 4TRLOU 单元的 SRAM 蝴蝶曲线图 (b) 使用逻辑门的 SRAM 蝴蝶曲线图	22
图 3-5 使用 4T RLOU 单元的 SRAM 版图	23
图 3-6 HSNM 测试电路	24
图 3-7 HSNM 仿真结果	24
图 3-8 (a) RSNM 的测试电路 (b) RSNM 的蒙特卡洛仿真结果	25
图 3-9 WM 测试电路图	25
图 3-10 WM 蒙特卡洛仿真结果	26
图 3-11 镜像加法器电路图	28
图 3-12 14T 的传输门型加法器	28
图 3-13 28T 与 16T 混合的逐级进位加法器树.....	30
图 3-14 (a)符号扩展电路 (b) 半加法器电路图 (c) 行波进位加法器结构	31

图 3-15 基于 16T 的加法器树结构图	32
图 3-16 混合华莱士树加法器结构图	33
图 3-17 华莱士树的符号扩展电路	34
图 3-18 (a) 28 管全加器单元版图 (b) 16 管全加器单元版图	35
图 3-19 全加器所有输入模式的功耗仿真	36
图 3-20 全加器所有输入模式的延时仿真	36
图 3-21 28T 与 16T 混合的加法器树版图	37
图 3-22 28T 与 16T 交替的加法器树延时与功耗仿真结果	38
图 3-23 16T 全加器最低输入电压仿真	39
图 3-24 16T 加法器树与方案一加法器树延时仿真	39
图 3-25 带缓冲器的 16T 加法器树延时仿真结果	40
图 3-26 全 16T 加法器树版图	40
图 3-27 16T 加法器树的延时与功耗仿真图	41
图 3-28 方案一与方案二性能指标对比	41
图 3-29 混合华莱士树结构的加法树延时及电流仿真	42
图 3-30 65nm 工艺下加法器树延时与功耗仿真	43
图 4-1 存算一体架构设计图	45
图 4-2 SRAM-CIM 阵列架构	46
图 4-3 输入控制模块基本单元	48
图 4-4 译码器电路图	48
图 4-5 灵敏放大器电路图	49
图 4-6 读写自适应模块结构示意图	50
图 4-7 累加器结构框图	51
图 4-8 累加器运算时序图	52
图 4-9 汉明距离运算结构图	53
图 4-10 SRAM 阵列延时及功耗仿真	55
图 4-11 累加器仿真时序图	56
图 4-12 存算架构版图	57
图 4-13 读写时序仿真图	58

图 4-14 存算架构乘加仿真时序图.....	58
图 4-15 XOR 操作时序仿真图	59
图 4-16 存算架构宏内各模块功耗占比.....	59
图 4-17 两种存算架构的性能对比.....	60
图 4-18 存算架构的算力与能效比结果.....	60

表 目 录

表 3-1 全加器真值表.....	27
表 3-2 行波进位加法器符号扩展电路的真值表.....	30
表 3-3 华莱士树的符号扩展电路真值表.....	34
表 3-4 16T 与 28T 全加器延时与功耗对比.....	37
表 4-1 可配置运算单元控制信号表.....	47
表 4-2 异或逻辑输出输出真值表.....	53
表 4-3 存内计算宏性能指标对比.....	61

第1章 绪论

1.1 研究背景与意义

近年来，人工智能（Artificial Intelligence，AI）技术尤其是以深度学习为核心的统计学习方法，已迅速成为主流。OpenAI 的分析显示，自 2012 年以来，AI 对算力的需求每 3.5 个月翻一番^[1]。这种不断增长的算力需求对现有的处理器技术提出了重大挑战。如图 1-1 所示，传统的冯·诺伊曼架构在计算时需要频繁地在存储器和计算模块之间传输数据，导致大量性能和能耗被用于数据传输和读写操作^[2]。然而，处理器性能的增长速度，遵循摩尔定律呈指数级增长，远远超过了存储器的发展速度^[3]。这种处理器与存储器之间的速度不匹配问题，逐渐成为满足高速计算需求的瓶颈，被称为“内存墙”和“功耗墙”问题^[4,5,6]。

因此，为了满足日益增长的高速计算需求，就需要开发新型的计算架构和处理器设计。目前，学术界和工业界都在探索如神经网络处理器（NPU）和量子计算等非传统的计算架构，这些新架构旨在提供更高效率的计算能力和能源效率，以应对现代 AI 技术对计算性能的严峻要求。

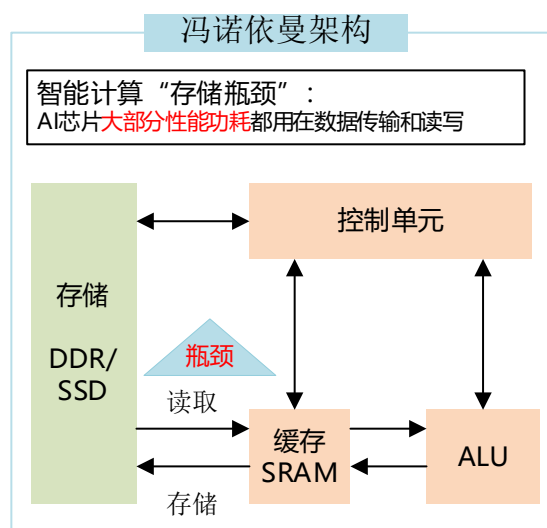


图 1-1 传统架构的存储瓶颈

存算一体（Computing-in-Memory，CIM）架构应运而生，这种架构最早由 Willian Kautz 在 1969 年提出^[7]，如图 1-2 所示，这种架构指的是将存储和计算功能集成在统一芯片或系统中的一种技术，使用存算一体技术有利于提高运算速度，减少数据搬运

的能耗，提高系统的稳定性及可靠性。这种技术可以在许多领域中得到应用，例如云计算、人工智能、物联网等。云计算可以通过存算一体技术来提高计算和存储的效率，同时也可以节约能源和成本。而物联网可以通过存算一体技术实现数据采集和处理的一体化，降低了系统的复杂度和功耗。总的来说，随着存算一体技术的不断成熟和发展，相信其在各个领域中的应用将会越来越广泛。

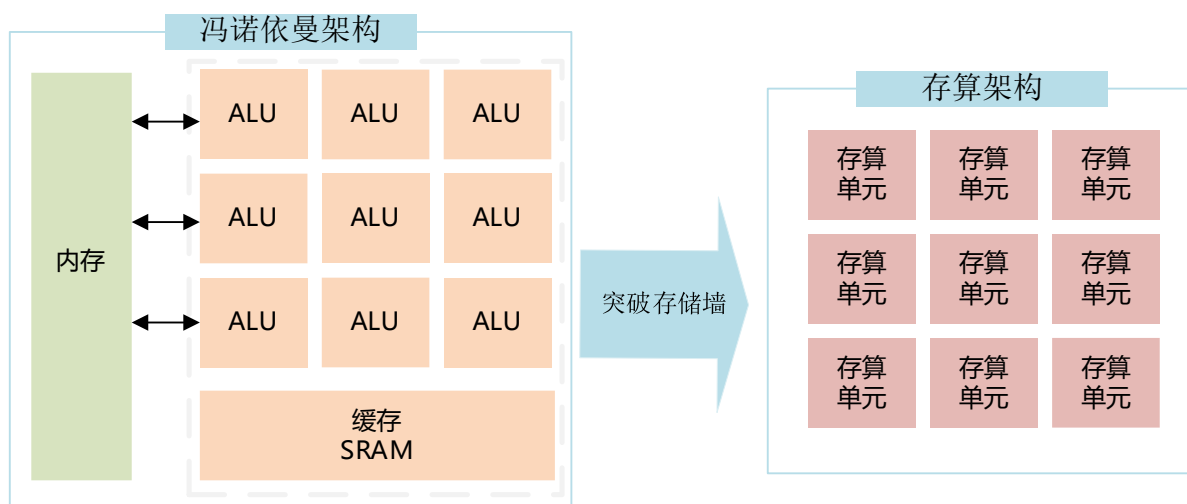


图 1-2 存算架构示意图

1.2 国内外研究现状

存算一体架构是一种新型的计算机体系结构，其核心思想是将存储与计算相结合，降低数据在内存与处理器之间传输的功耗和延时。目前国内外对存算一体技术的研究都比较活跃，大多集中在算法和芯片方面，本文主要总结芯片方面的研究。目前全球存算技术还在快速发展的阶段，英特尔、三星等芯片巨头都在探索这一技术，国内也有一批存算企业兴起。

在存算架构中，“存”代表存储器，这种计算体系结构的存储器主要分为两大类^[8]：易失性存储器和非易失性存储器（Nonvolatile Memory, NVM）。易失性存储器的主要特点是数据在断电时会丢失，这对于临时性的、计算过程中产生的数据来说是合适的，其中掉电易失性存储器包括静态随机存储器（Static Random Access Memory, SRAM）和动态随机存储器（Dynamic Random Access Memory, DRAM）。由于 SRAM 的工艺成熟且应用落地快，基于 SRAM 的存算架构在业界受到广泛关注。本节将根据存储器是否为 SRAM 来介绍国内外的研究现状。

1.2.1 基于非 SRAM 的存内计算研究现状

2019 年^[9], 旺宏电子提出了一种基于 3D NAND 闪存 (flash) 的高密度、高带宽、低功耗的非易失性存内计算 (Nonvolatile Computing-in-Memory, nvCIM) 的设计方法, 用于深度学习神经网络 (Deep-Learning Neural Network, DNN) 的推理。文中提出的设计方法优化了 3D NAND 闪存, 以实现高效的 DNN 计算。优点包括高密度、高带宽、低功耗, 适合大型神经网络的边缘计算。然而, 这种架构可能面临非理想的单元变异、噪声、漂移等挑战, 需要适当的校准技术来确保计算精度。

2021 年, 德州大学提出了一种基于 eDRAM 的存内计算架构^[10], 用于实现自适应数据转换器和电荷域计算。这个设计允许在 eDRAM 阵列中直接执行模拟 DNN (深度神经网络) 计算, 例如数据转换、点积、平均池化和 ReLU 激活。然而, 由于 DRAM 技术相对于 SRAM 的成熟度较低, 其应用范围会受到限制。与 DRAM 相比, SRAM 存储单元的面积虽然较大, 但是不需要周期性的刷新操作就可以保持数据的稳定性, 可以降低功耗。

2022 年, 三星电子在顶级学术期刊 Nature 上发表了全球首个基于磁性随机存储器 (Magnetic Random Access Memory, MRAM) 的存内计算论文^[11], 克服了 MRAM 低电阻特性, 采用新的“电阻总和”架构, 代替了传统的“电流和”架构, 从而解决了 MRAM 低电阻的问题。但是将 MRAM 阵列与现有的系统架构 (如 AI SoC) 集成可能面临技术挑战, 需要确保与多种数据转换器和数字电子设备的兼容性。

2023 年, 东南大学提出了一种基于 28nm 工艺的 2Mb STT-MRAM (自旋转移矩磁随机存取存储器) 的存算宏^[12], 他们采用了改良的位单元设计, 并实现了 22.4 - 41.5TOPS/W 的 AI 推理性能。这种架构的优点包括高能效比、快速读取能力, 以及适用于 AI 推理应用。然而, 它也存在缺点, 如制造复杂性和成本较高, 以及相对于其他成熟存储技术可能存在的可靠性挑战。

总的来说, 基于非 SRAM 的存算架构大多存在技术成熟度不高和可靠性问题。这些架构由于非 SRAM 存储器在访问和处理数据时通常需要更长的时间, 因此在数据处理速度和计算效率方面表现较差。

1.2.2 基于 SRAM 的存内计算研究现状

SRAM 由于具有读写高速、结构可靠等优点, 因此被广泛地用于存内计算领域。在基于 SRAM 的存内计算架构被分为两个方向, 一种是模拟存算, 一种是数字存算。

模拟存算主要是利用电流或者电压等连续信号对数据进行运算，而数字存算则是利用数字逻辑对数据进行运算。

首先介绍模拟存算，2018年，Intel的loihi芯片中用到了基于SRAM的存内计算的技术，用电流信号模拟神经脉冲信号进行累加，从而大幅提高了脉冲神经网络的计算速度^[13]，但作为一种研究级芯片，它仍然面临着将神经形态计算从实验室研究推向商业可行技术的挑战。九天睿芯研究神经拟态感存算一体芯片，第一颗感存算一体芯片ADA100已于2021年回片^[1]，这标志着在感知和计算结合的领域取得了重要进展。神经拟态感存算一体芯片的设计通常结合了神经拟态计算和存储功能，实现了感知、存储和处理的一体化。这种设计可以在处理感知任务时更加高效，并且在能效方面有显著的优势。

国立清华大学司鑫课题组在2019年发表了一篇关于基于分裂式字线8T SRAM存算一体的文章^[14]，该架构使用数模转换器（Digital to Analog converter, DAC）将数字信号转换为模拟信号，然后通过电流进行累加操作。随后，采用模数转换器（Analog to Digital converter, ADC）将累加后的电流转换回数字信号以输出最终结果。此后三年^[15, 16, 17]该团队的成果也围绕模拟存算展开，2023年发表了一篇混合域浮点数存算一体架构的论文^[18]。这个芯片实现了16.2-70.2TFLOPS/W的高效能，通过结合数字和模拟域的计算，优化了浮点运算的能效和准确性。这种设计特别适用于需要高能效和高推理精度的先进AI边缘设备。

模拟存算因其在能效和速度方面的优势，特别适用于需要快速处理且对能耗敏感的场景，如神经形态计算和图像处理。然而，模拟存算也存在缺点，包括易受噪声影响和输出精度较低。相比之下，数字存算提供了更高的稳定性和精确性，适合于更广泛的应用，如传统数据处理和高性能计算。

2022年，苹芯科技和北京大学在ISSCC中提出了一种高效的无ADC架构SRAM存内计算加速引擎^[19]，阵列大小为32Kb，可实现多种逻辑运算并支持多种不同精度，优点是使用面积小且功耗低。这项技术的重要特点包括高能效比与面效比、动态逻辑计算电路（DCC）、可重构的本地处理单元（RLPUs）。该设计采用动态逻辑计算电路替代了传统的ADC或CMOS静态逻辑，从而达到了27.38TOPS/W的高能效比，同时实现了高达1.041Mb/mm²的面效比，这在国际上处于领先水平。但同时，动态逻辑计算电路虽提高了能效，却可能会增加电路设计的复杂性，这对于制造过程和维护可能构

成挑战。

2022 年,亚利桑那州立大学的 Deliang Fan 教授团队也在研究存内计算,他们发表了一篇论文^[20],提出了一种基于 8T SRAM 的可实现多种逻辑运算的计算架构。这个设计实现了在单个存储周期内完成一整套布尔逻辑操作(包括 2 输入和 3 输入的 NOR/NAND/XOR、乘法和加法)。PSRAM(Programmable SRAM)提供了存储器计算平台所需的可编程性,适用于各种应用,如并行向量操作、神经网络和数据加密。芯片在 1.2V 电压下实现了 1.2 TOPS 的峰值吞吐量和 34.98 TOPS/W 的能效。但是由于复杂的逻辑切换电路,这种结构只能使用分时复用的策略,降低了芯片的吞吐量。

2023 年,复旦大学刘琦教授团队在 ISSCC 会议上展示了一款名为“天溪”的存算一体芯片^[21],专为 Transformer 神经网络设计。该芯片采用基于 10T-SRAM 的存内计算和数字处理技术,通过蝶形数据分配网络高效处理稀疏数据,大幅提高了系统效能和能效。在处理不同神经网络任务时,其系统效能分别达到 11.83 TOPS/W、53.83 TOPS/W 和 25.22 TOPS/W,比现有 Transformer 网络加速器高出 3.2 至 9.7 倍。但是这一技术可能需要进一步开发以更好地兼容各种类型的神经网络架构。

台积电作为最早研究基于 SRAM 存算芯片的企业之一,其成果及经验也远超其他企业。台积电在 2021、2022、2023 的 ISSCC 上均发表了关于存算一体的文章,其中既有数字存算又有模拟存算,从仅完成乘加操作的模拟存算^[22]到数字存算^[23],再到可进行反馈更新的全数字存算^[24]。这些成果表明,台积电在推动存算一体技术,特别是数字存算领域的创新和发展方面处于行业领先地位。台积电的研究不仅展示了存算技术在提高计算效率和降低能耗方面的巨大潜力,还体现了其在满足日益增长的数据处理需求,特别是在人工智能和边缘计算应用中的重要性。

存内计算技术在人工智能、大数据分析、物联网等领域展现出巨大的应用潜力。通过在内存阵列中直接执行计算任务,CIM 技术能显著提高计算效率,减少数据传输所需的时间和能耗。这种技术特别适用于处理大规模数据和复杂计算,有助于推动智能设备和高效能计算系统的发展。未来,随着 CIM 技术的进一步研究和优化,我们可以预期它将在这些领域发挥更加重要的作用。

1.3 论文的研究内容与创新点

在当前存算一体技术领域,虽然存储与计算的结合已经显著提高了运算效率,并

且广泛应用于神经网络加速，但仍面临能效比不高和运算逻辑单一、应用场景受限的挑战。基于此场景下，本文提出了一种高能效比、可配置的存算架构。该架构设计了 4T 计算单元以实现多种逻辑运算，并提出了改进的全 16T 加法器树结构，相较于 28T 的加法器树，其功耗更低，面积更小。在 28nm 工艺下实现该架构，并对其性能指标进行了测量和分析。其主要内容与创新点如下所示：

（1）可配置的逻辑运算单元

本文提出了一种可以配置的逻辑运算单元，根据控制信号可以完成 AND，OR，XOR 等布尔运算。目前存在的存算一体架构能够完成的运算逻辑比较单一，本文设计了一种可配置的电路结构，使得存算架构能够应用于更多场景中。这种电路结构由两部分构成，一部分是输入控制电路，与信号线的驱动电路结合在一起，由于信号线本就需要驱动模块，因此这部分的控制电路面积及功耗可以忽略；还有一部分置于 SRAM 阵列中，每一个 SRAM 单元只需要额外使用四个 MOS 管，便可完成电路运算逻辑的切换。

（2）改进后的 16T 加法器树结构

加法器树由加法器单元以二叉树的方式搭建，传统加法器单元由 28 个 MOS 管组成，如使用 28T 的加法器搭建加法器树，电路的面积及功耗会比较大。因此，本文提出了一种使用 16T 加法器的电路架构，相比于 28T 的加法器，这种架构在功耗与面积上有很大优势。

1.4 论文结构

论文分为六章，内容如下分布：

第一章为绪论，首先说明了存算一体架构出现的必要性与应用场景，简单介绍了存算一体的基本架构；接着阐述了存算一体架构在国内外的研究现状，分析了不同架构的优缺点；最后写明了本篇论文的研究内容与创新点。

第二章详细介绍了 6T SRAM 的电路原理、性能评估指标以及外围电路，同时讨论了存算一体架构的分类以及在存算一体架构中的计算操作，包括卷积操作和汉明距离的计算。

第三章详细介绍了本文提出的两个关键模块：一是基于 SRAM 的可配置运算逻辑存算单元，并对存算单元进行了后仿真验证；二是为提高存算架构的能效比而提出的

三种不同的加法器树设计。这些设计包括 28T16T 加法器树、改进后的全 16T 加法器树以及混合华莱士树加法器树。文中通过对这三种加法器树进行前仿真与后仿真分析，从功耗、速度、能效比等多个方面进行了对比分析。

第四章主要提出了一种基于 SRAM 的可配置的存算一体架构，该架构可以完成乘加操作和布尔逻辑运算（与、或、异或逻辑），并对其中的每个模块进行介绍和仿真。还具体分析了架构的应用场景，最后对论文提出的架构进行仿真验证，与其他论文的性能指标进行比较分析。

第五章总结了主要工作并对未来的工作进行展望。

第2章 基于 SRAM 的存算一体电路及工作原理

2.1 基于 SRAM 的存储架构

本文提出的存算架构基于 SRAM 进行设计。在这一部分中,我们将重点介绍 SRAM 的基本架构。如图 2-1 所示,经典的 SRAM 阵列包括存储阵列、预充电电路、译码电路、时序控制电路以及输入/读出电路,其中存储阵列使用传统的 6T SRAM 单元,本节将基于这一阵列对存储系统进行详细的介绍。

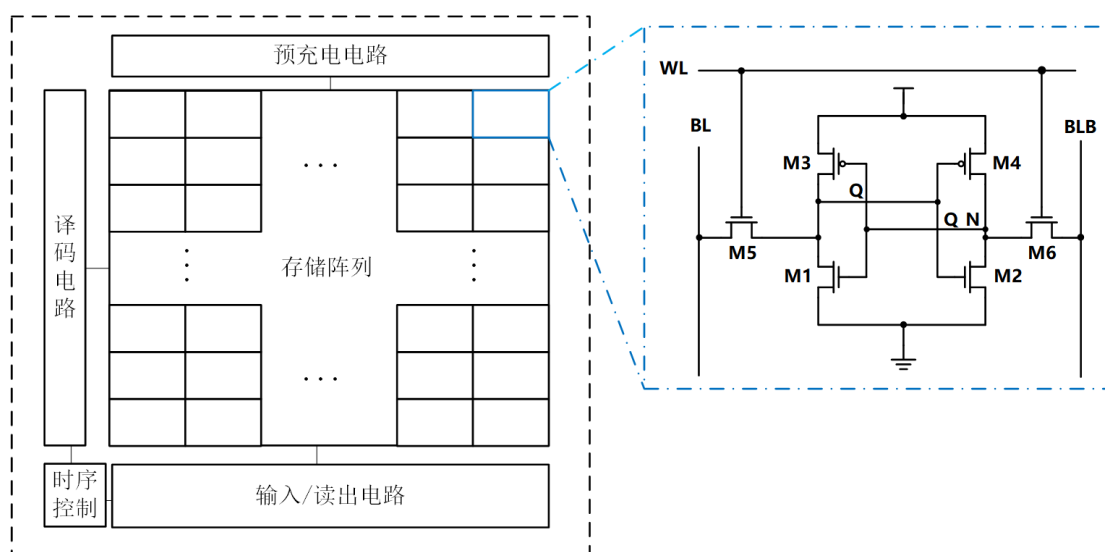


图 2-1 SRAM 阵列电路图

2.1.1 SRAM 存储阵列

2.1.1.1 SRAM 单元

SRAM 阵列是由一系列静态随机存取存储器 (SRAM) 单元组成的矩阵结构。如图 2-1 所示, SRAM 单元由 6 个 MOS 管构成, 其中, M1-M4 组成一个 latch 锁存器, 用于保存数据, M5、M6 是用于传递数据值的选通管。WL 是字线的控制信号, BL 和 BLB 是位线的控制信号, 而 Q 与 QN 是存储在 SRAM 内的两个相反的值, 其中节点 Q 表示的是写入或者需要读出的数据, 而 QN 表示的数据与写入或者需要读出的值相反。当对应的 SRAM 单元的 WL 字线打开时, 该单元就是被选中的单元, 通过 BL 跟 BLB 传递数据, SRAM 单元的 MOS 便会工作, 完成数据的写入或读出。

图 2-2 表示了数据写入的过程, Q 的值主要由 M3 与 M5 相互之间的强弱决定, 而 QN 的值主要取决于 M2 和 M6 相互之间的强弱。一般情况下, 由于 NMOS 通道的电子迁移率更高, NMOS 的驱动能力比 PMOS 的驱动能力强, 因此, 写操作时的关键

是写“0”操作。假设6T SRAM中存储的值为“1”，即Q节点为“1”，QN节点为“0”，需要写入的新的Q节点的值为“0”。在字线WL开启之前，BL上的值将被放电到VSS，BLB上的值将被充电到VDD。写操作时WL线打开，M5、M6被选通，由于BL被放电到VSS，原来的Q值翻转，M4管打开，QN值改变，相互耦合的锁存器使得数据更加稳定，写“0”操作完成^[25]。其中涉及到的一个写容限（Write Margin）的问题，写容限表示BL上可以达到逻辑0的最大电压，也就是当BL达到某一电压后，SRAM单元无法完成写零的操作。

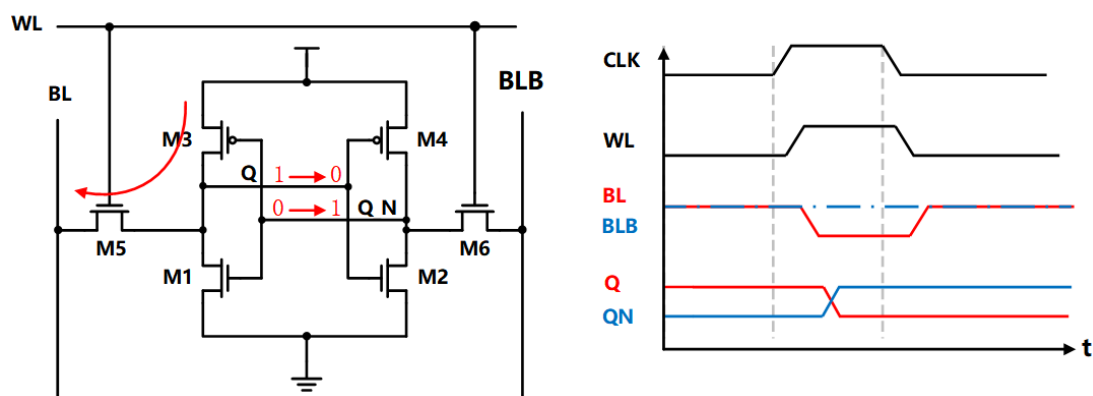


图 2-2 6T SRAM 单元电路图及写入波形

图 2-3 表示了数据读出的过程以及信号的波形，与写操作类似，6T SRAM 单元的读出数据的正确性也依赖于 MOS 管的驱动能力，其关键是读“0”的过程，BL 上的值主要由 M5 及 M1 之间的强弱决定。在读操作时，BL 与 BLB 将会被预充电到 VDD，接着开启 WL，M5 和 M6 被选通，此时 QN 线上的电平为“1”，M1 打开，BL 线上的电荷便会通过 M5-M1 通路接到 VSS 放电，这会使得 BL 上的电压降低；而 BLB 通过 M4-M6 通路与 VDD 相接，使得 BLB 的电压保持不变。最后，通过灵敏放大器比较 BL 与 BLB 线上的电压差，便可以读出 SRAM 单元存储的值。其中要注意的是，BL 线上放电时，Q 值为 M1 与 M5 的分压，因此会随着 BL 上电荷的减少出现一个小电压抬升^[26]（voltage bump），如果这个电压抬升过大，会使 SRAM 内存储的值翻转出现错误，这一现象即为读干扰（read disturb issue）。

为了保证 SRAM 存储值的正确性，Latch 的双稳态必须保持稳定，静态噪声容限（Static Noise Margin, SNM）便是为了衡量这一性能，这也是 SRAM 读性能的重要指标。SNM 的测量方法是固定 BLB 的值，逐渐扫描 BL 上的电压，观察 Q 上的电压，整合出波形图，这也就是“蝴蝶曲线”（butterfly curve）。

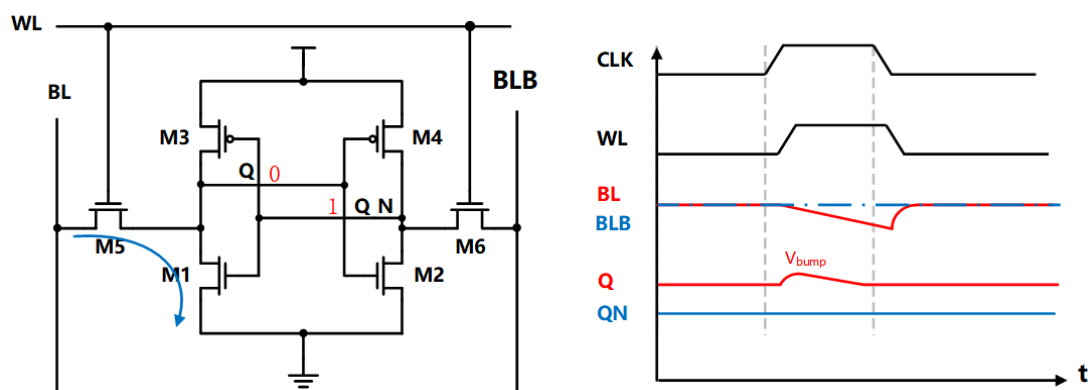


图 2-3 6T SRAM 单元读操作及其波形图

2.1.1.2 影响静态噪声容限的因素

如图 2-4 所示，是 SRAM 用于测量静态噪声容限的“蝴蝶曲线”。静态噪声容限是衡量 SRAM 存储单元的稳定性和噪声容忍度的一个重要指标。它表示在理想条件下，SRAM 存储单元的读写操作能够容忍的噪声水平。具体而言，静态噪声容限是指在 SRAM 存储单元中，当读和写的信号边界发生偏移时，存储单元依然能够正确地保持数据。SNM 通常以电压为单位，表示为在不发生错误的情况下，读和写操作的电压边际。较大的静态噪声裕度表示 SRAM 存储单元对于噪声的容忍能力较强，提高了存储系统的可靠性。因此我们需要努力优化 SRAM 的结构和参数，以获得更大的静态噪声容限。

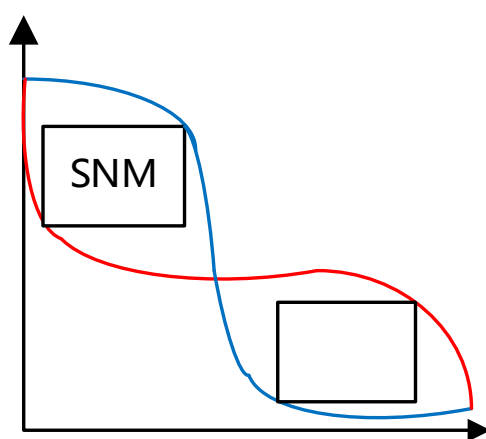


图 2-4 SRAM 的蝴蝶曲线

要获得更大的静态噪声容限，可以通过以下方法优化 SRAM 的结构和参数：

1. 电压裕度设计：通过在设计中给予电源电压一定的裕度，可以增加 SRAM 存储单元的稳定性。使用更高的电源电压或者更低的工作频率可能会提高静态噪声裕度，但需注意功耗和性能之间的平衡。

2. 调整存储节点的尺寸：调整存储节点的尺寸可以影响存储单元的电荷存储和读写性能，从而影响静态噪声裕度。在存储节点尽量使负载对称，确保晶体管间的良好匹配和优化布局。
3. 优化电源噪声：降低电源噪声可以提高SRAM的噪声容忍度。采取良好的电源电感和电容设计，以减少电源波动和噪声。

2.1.2 SRAM 存储阵列的外围电路

2.1.2.1 预充电电路及时序分析

如图 2-5 所示，预充电电路由三个 MOS 管构成，M1 与 M2 分别跟 BL、BLB 两条位线相连，用于控制位线的预充电。M3 连接 BL、BLB，预充电的时候用于连接两点电位，使两点电位平衡。Precharge_n 为预充电控制信号，低电平有效。

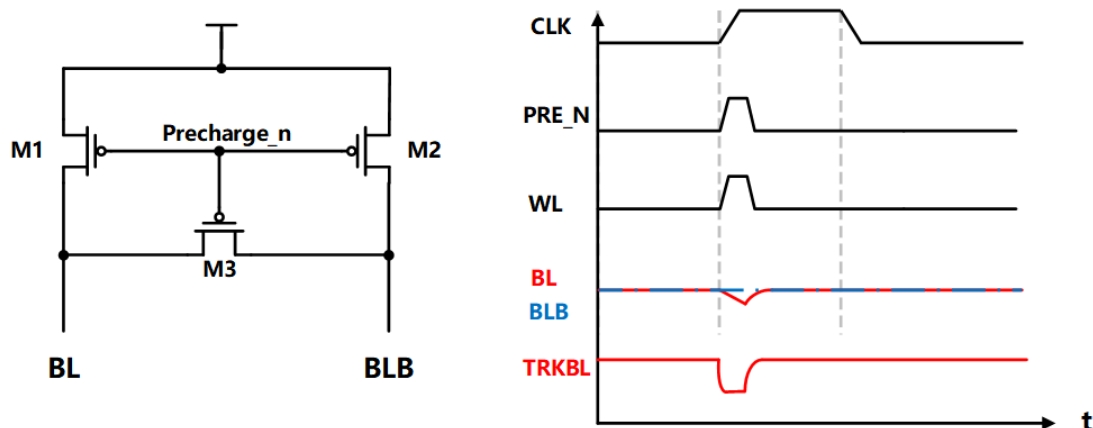


图 2-5 预充电电路及时序

在进行数据读出时，预充电电路的时序是影响 SRAM 电路功耗的重要因素之一，电路的读写越快，预充电的时间越长，电路需要的静态功耗越小。因此，当写入读出结束时，预充电电路便可以开始工作进行预充电，保持 BL 与 BLB 线上的电压，降低电路的功耗，在这里可以设计一个自适应（self-timing）的读写时序控制电路。

如图 2-6 所示，self-timing 电路会用到一个冗余（dummy）阵列，dummy 阵列一般与存储阵列放在一起，这是为了更好地模拟存储阵列的环境，dummy 阵列里的 SRAM 单元与存储阵列中设计的 SRAM 单元一样，其不同之处是 BL 线的连接，在 dummy SRAM 阵列中，BL 线被命名为 TRKBL（Track BL），TRKBL 只连接了部分 SRAM 单元，这是为了模拟 BL 线上的电容大小，具体连接几个 SRAM 单元需要通过仿真决定，也就是在写操作时，需要仿真存储阵列里的 SRAM BL 的放电时间，如图 2-5 的波形图所示，一旦写入操作完成，TRKBL 降为低电平，通过逻辑电路拉高 precharge 信号，

进行预充电。当然，在读操作时，采样电路也需要在这段时间内完成采样。这样，便可以降低功耗，完成SRAM电路读写的时序控制。

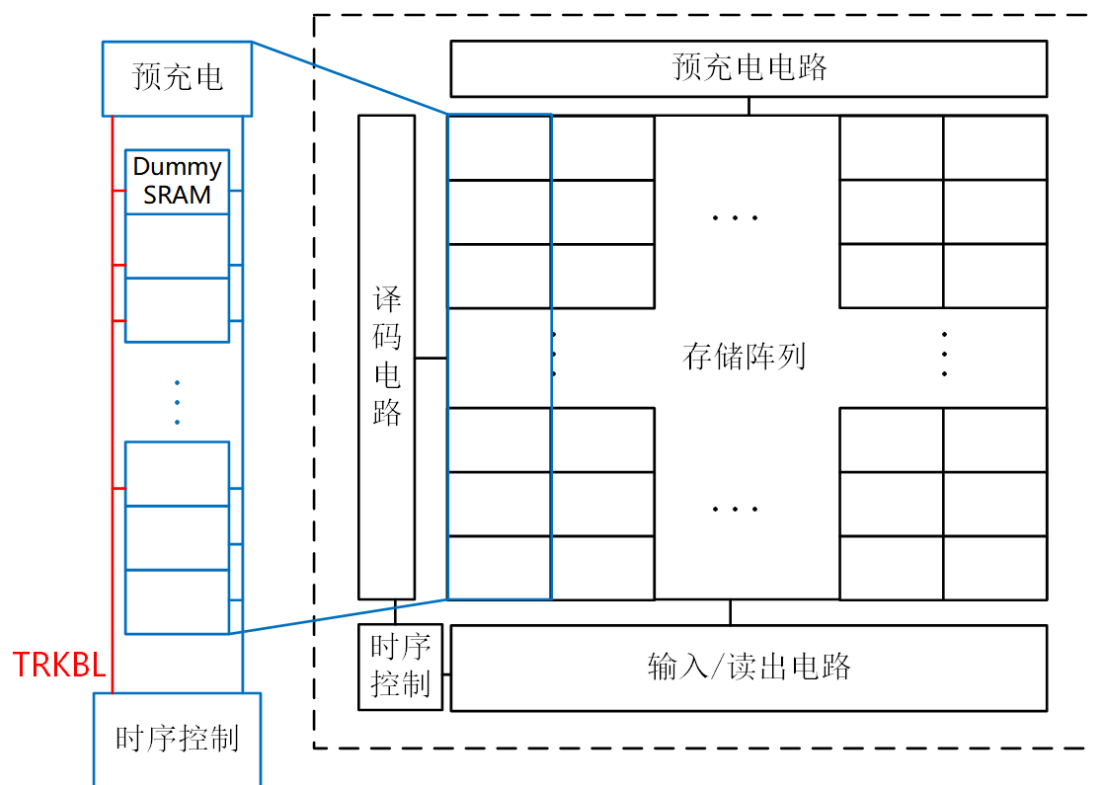


图 2-6 self-timing 电路结构图

2.1.2.2 译码器电路

当存储阵列较小时，可以通过控制信号线对阵列进行直接控制，但是当阵列逐渐增大，直接使用控制信号进行控制的方法就比较困难，并且大量控制信号也不利于芯片的封装，这就需要用到译码器电路，使电路更加简洁且易于操作，使用译码器电路的另一个好处是可以使电路的输入输出端口更少，降低设计成本。

译码器一般由逻辑门实现^[27]，其中常使用到的有 2-4 译码器和 3-8 译码器，如图 2-7 所示。当地址位数较多时，译码器的设计可能会遇到扇出（fan-out）的问题，即一个逻辑门的输出需要驱动过多的输入门。在这种情况下，通常采用二次译码的方法来降低电路的扇出。二次译码指的是使用两个或更多级的译码器，其中第一级译码器的输出成为第二级译码器的输入。这种设计可以有效地分散负载，减少单个逻辑门需要驱动的输入数，从而提高整个系统的可靠性和效率。

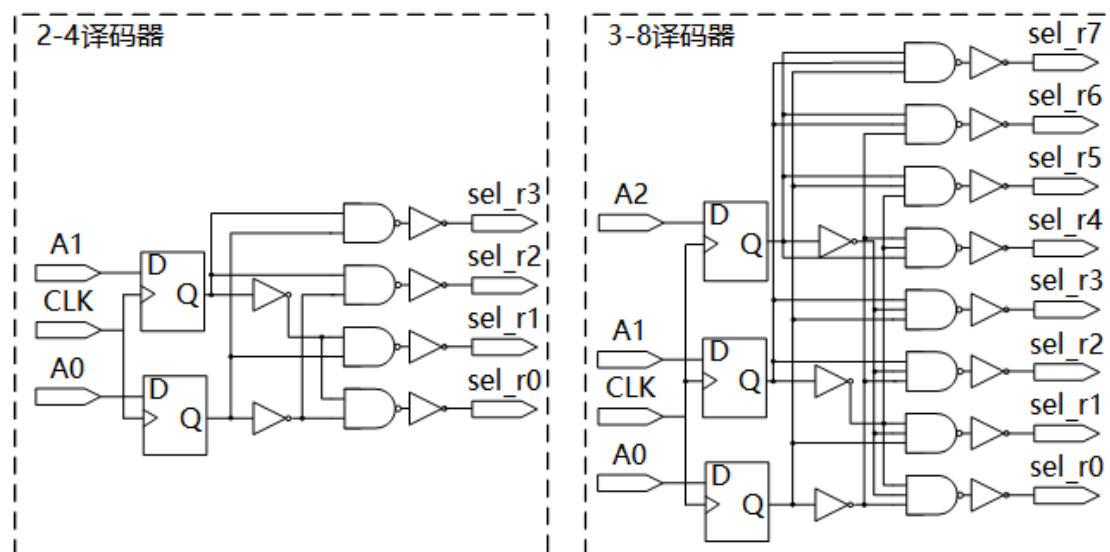


图 2-7 2-4 译码器与 3-8 译码器电路图

2.2 基于 SRAM 的存算一体技术简介

目前评价存算一体架构的主要性能指标有：1. 所支持的运算类型和计算的精度，运算类型包括布尔逻辑（Boolean）运算、单比特乘加运算、多比特乘加运算，计算精度主要指的是可输入以及权重信息的比特数。2. 架构的吞吐量（Throughout）大小及能效比（Energy Efficiency），算力大小指的是单位时间内可完成的运算次数，能效比表示每秒内计算次数与功耗的比值。

2.2.1 存算一体架构分类

存内计算技术旨在克服冯·诺依曼瓶颈，通过在存储器中集成计算功能，实现计算吞吐量和能效的可持续提升^[28]。如图 2-8 所示，目前存算一体架构根据计算方式主要分为两类：数字存算^[22,29,30]与模拟存算^[31,32,33,34]。这两种类型的架构在计算操作上采用截然不同的方法，以满足各自的应用需求。模拟存算存算架构依赖于电压或者电流来执行计算操作，计算过程通常以连续的电压或者电流完成，模拟存算的主要优点是低功耗和高速，特别是在需要连续数据处理的应用中，如模拟信号处理或某些类型的图像处理，模拟存算表现出色。但模拟存算面临着精度低和电路复杂的问题。

相比之下，数字存算采用数字逻辑门进行计算，数据以高精度的数字形式表示。这种架构能够处理各种复杂的算术和逻辑运算，包括加法、逻辑与、或、非运算等，甚至能够实现更复杂的计算任务。数字存算的最大优势在于其极高的灵活性和精度，以及对输入数据规模的不敏感性，使其具有很强的可扩展性。因此，数字存算被广泛应用于

需要高精度和灵活计算的领域，如机器学习、大数据分析和复杂算法的实现。

综合考虑，本论文选择了数字存算技术。这是因为数字存算不仅在精度上占优，而且在处理复杂计算任务时显示出更高的灵活性和可扩展性。随着技术的发展，数字存算有潜力在未来的计算任务中发挥更重要的作用，尤其是在需要高性能和高精度计算的应用中。

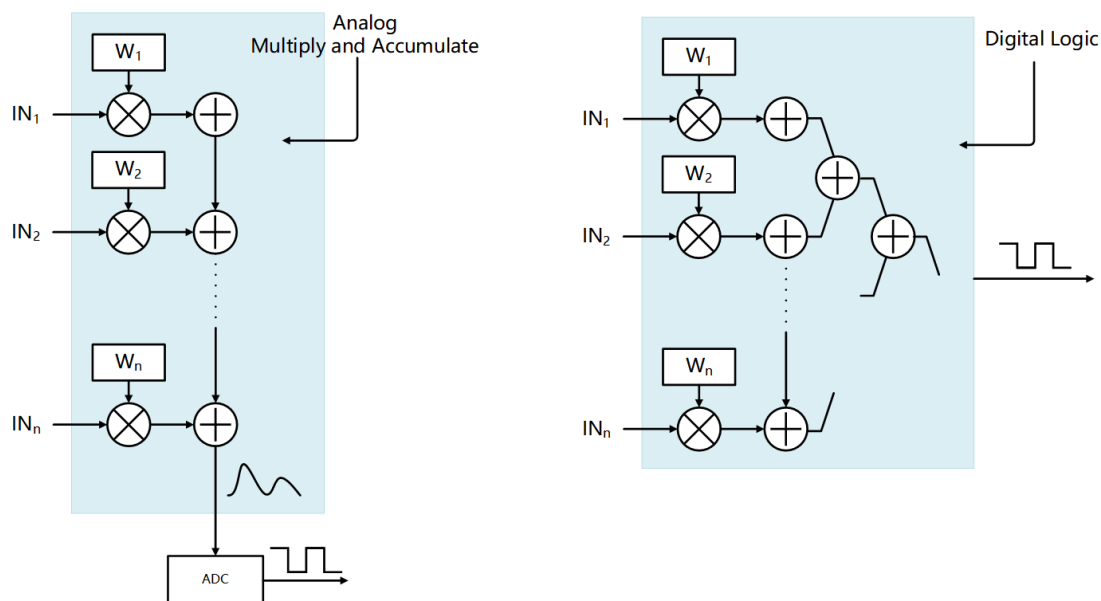


图 2-8 模拟存算与数字存算

2.2.2 卷积操作概述

卷积操作是许多领域中常用的数学工具，特别是在信号处理和图像处理中。在这些应用中，卷积通常用于滤波、特征提取等。卷积操作的基本形式可以表示为两个函数（或信号）的积分，其中一个函数被翻转并移动。

对于连续函数，卷积定义为

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau \quad (2-1)$$

这里 f 和 g 是两个连续函数， t 是变量， τ 是积分的变量。

对于离散函数，尤其是在数字信号处理和计算机视觉中，卷积公式变为：

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m] \cdot g[n - m] \quad (2-2)$$

在这里， f 和 g 是离散序列， n 和 m 是整数。

在图像处理中，卷积通常用于应用滤波器（如边缘检测、模糊等）。在这种情况下，卷积操作可以表示为图像（作为矩阵）和一个小的矩阵（称为卷积核或滤波器）之间的操作。对于二维图像卷积，公式为：

$$I'(x,y) = \sum_{i=-k}^k \sum_{j=-k}^j I(x+i,y+j) \cdot K(i,j) \quad (2-3)$$

其中是 I 原始图像， K 是卷积核， I' 是卷积操作后的图像， x,y 是像素位置， i,j 是相对于当前像素的卷积核位置。在深度学习中，卷积神经网络（CNN）使用类似的卷积操作来提取输入数据（如图像）的特征。

卷积操作通常可以通过图表来表示，如图 2-9 所示，其中包括原始图像、卷积核、输出三个图层。原始图像表示为一个较大的矩阵，卷积核是一个较小的矩阵，它在原始图像上“滑动”，卷积操作的输出通常是另一个矩阵，其大小取决于原始图像的尺寸、卷积核的尺寸和步长（stride）。

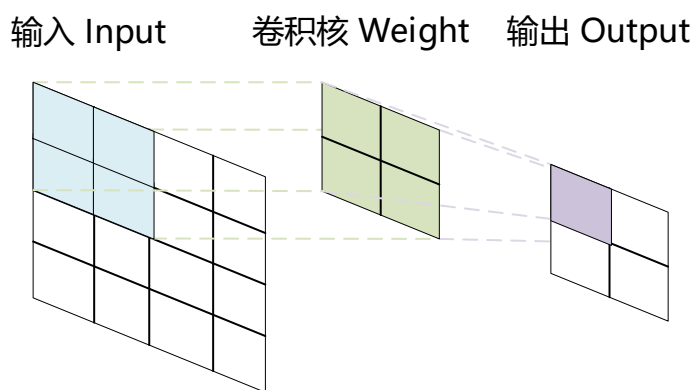


图 2-9 卷积操作示意图

存算一体架构中，进行卷积操作通常遵循以下步骤：首先，将输入数据（例如图像）和卷积核加载到存储阵列中。在 CIM 架构中，这些数据可以直接存储在存储单元（如 SRAM 单元）中。当输入数据和卷积核被加载到存储阵列后，可以同时多个数据点执行卷积操作。将卷积核中的每个元素与输入数据中相应位置的元素相乘，然后将这些乘积求和，就可以得到卷积结果。

在存算一体架构中实现卷积操作，提高了计算效率并减少了数据在存储器和处理器之间的传输。CIM 利用存储器阵列（如 SRAM）内部的计算能力，直接在数据存储的位置执行计算任务，这对卷积神经网络特别有利，这是因为卷积操作涉及大量重复访问相同的图像区域，CIM 可以利用这种数据局部性，减少数据移动，直接在数据存储的地方进行卷积计算。并且，CIM 允许在存储阵列的多个位置同时执行卷积操作，实现高效的并行处理，显著降低能耗。

具体来说，CIM 架构可以被配置为直接在存储阵列内进行卷积核的应用，执行像素乘积并累加，从而实现图像的特征提取。这种方法特别适合处理大规模图像数据，可以大幅提高卷积神经网络处理图像任务的速度和效率。

2.2.3 汉明距离的运算

汉明距离是衡量两个等长字符串之间差异的一个指标，特别是在比较二进制串时被广泛使用。它定义为在两个相同长度的字符串中，对应位置上不同字符的总数。

$$d_H(A, B) = \sum_{i=1}^n (A_i \neq B_i) \quad (2-4)$$

这里 A_i 和 B_i 是数组 A 和 B 在第 i 位置上的码元， n 是数组的长度，求和操作遍历所有位置，如果在位置 i 上 A_i 和 B_i 不同，则该项的值为 1，否则为 0。最终，所有这些值的总和就是 A 和 B 之间的汉明距离。

如图 2-10 所示，如果我们有两个字符串和，它们的长度相同，则和之间的汉明距离是在它们各自对应位置上不同字符的数量。例如，如果字符串在某个特定位置上是 '1' 而字符串在同一位置上是 '0'，那么这个位置就会对汉明距离贡献 1 点。将所有这些差异点加起来，就得到了两个字符串之间的汉明距离。

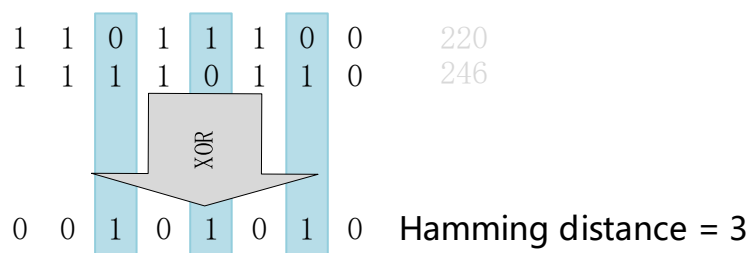


图 2-10 汉明距离计算示意图

在存算架构中，计算汉明距离通常涉及以下步骤：首先，将两组数据（通常是二进制串）加载到存储阵列中。在某些架构中，这可能意味着将一组数据存储存储在存储单元中，而另一组数据则作为输入送入计算单元。汉明距离的计算本质上是比较两组数据中对应位的差异。这通常通过异或（XOR）逻辑运算实现，因为异或运算能够有效地识别两个比特位是否不同（即一个为 0，另一个为 1）。在存算架构中，这个运算可以直接在存储单元中进行，这样可以显著减少数据在存储单元和处理器之间的传输，提高计算效率。异或运算的结果是一个新的数据集，其中每个位上的“1”表示原始数据集在该位置上不同。接下来，需要统计这个结果中“1”的数量，这通常通过累加器或计数器实现。最后，统计完异或结果中的“1”数量后，得到的数值即为两组数据之间的汉明距离。通过这种方式，存算架构能够高效地计算出汉明距离。

在信号处理中，汉明距离可以用来衡量信号间的相似度或差异度。在模式识别和机器学习中，它常用于分类算法中，以评估不同类别或标签之间的差异。例如，在文本

处理中，可以用汉明距离来比较两段文本的相似性；在生物信息学中，用于比较不同 DNA 或蛋白质序列的相似性。此外，在通信领域，汉明距离也用于编码理论，帮助识别和纠正错误。在这些应用场景中，CIM 架构计算汉明距离的能力意味着能够更快处理大量数据，同时降低能耗。这对于实时数据处理和在资源受限的环境下的计算尤其重要。此外，CIM 架构在这些应用中的使用可以提高整体系统的性能和可靠性。

2.3 小结

本章首先介绍了基于 SRAM 的存储架构，包括 6T SRAM 的电路结构以及工作原理，包括读操作、写操作以及静态噪声容限，同时介绍了 SRAM 存储器的外围架构包括译码器和预充电电路，接着介绍了存算一体的架构分类以及目前在存算中的两种计算操作：卷积操作和汉明距离计算。

第3章 存算一体技术中关键模块的设计

针对前文提出的存算设计的要点，本章节主要聚焦于存算架构中的两个关键模块：存算单元和加法器树模块。这一部分将详细介绍这两个模块的设计原理、工作机制以及它们在整个存算架构中的作用和重要性。

3.1 面向卷积神经网络的存内计算系统模型

面向卷积神经网络（Convolutional Neural Networks, CNN）的存算架构需要执行乘法（Multiply Accumulate, MAC）操作。这里提出了一种典型的基于 SRAM 的面向 CNN 存算一体架构的模型，这个模型为存算单元和加法器树的设计提供了基础条件，如图 3-1 所示，该模型包括 SRAM-CIM 阵列、时序控制电路、加法器树电路和移位累加电路。

在这个模型中，存算单元由一个 6T SRAM 和一个逻辑与门组成，负责完成权重与输入激活因子的乘法操作。其中，SRAM 阵列中存储的是权重信息，每组权重为 4 位，也就是一组权重信息需要 4 个 SRAM 单元。当有 256 个激活因子（ IN_n ）输入时，存储在 SRAM 阵列中的权重信息 Q 与 IN_n 进行与（AND）操作，即相乘操作。SRAM-CIM 阵列的输出结果是 256 组 4 位的中间积，这些数据将被送入加法器中。

加法器树的功能是将这 256 组数据进行相加，得到部分和。由于加法器一般由逻辑门组成，其规模随着阵列的增大而扩大，因此，在存算阵列的模型中，加法器树是影响存算阵列功耗和面积的重要因素之一。除此之外，在存算模型中，相加操作与相乘操作时在同一个周期内完成的，这就要求加法器树所用的延时尽量短，所以加法器树的设计在提高存算一体架构的能效比方面至关重要。

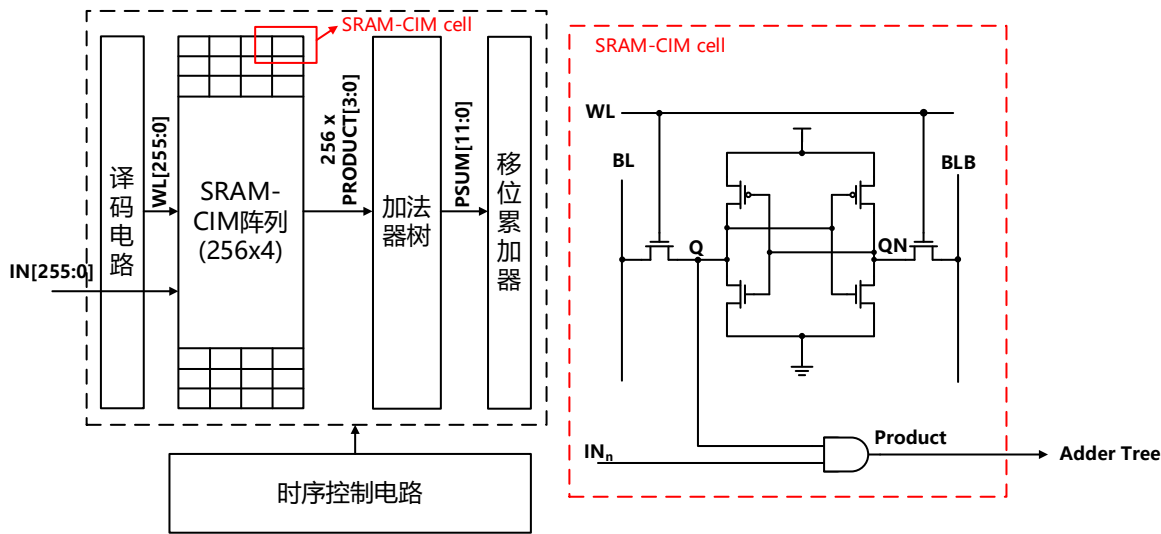


图 3-1 面向 CNN 的基于 SRAM 的存算一体架构

3.2 可配置逻辑运算的 SRAM 单元设计与分析

本文提出了一种基于 4T 可配置逻辑运算单元（Reconfigurable Logic Operation Unit,RLOU）的存算单元设计方案，如图 3-2 所示。存算单元由一个 6T SRAM 存储单元和一个 4TRLOU 计算单元组成，相较于上文提到的传统的以逻辑门为核心的 SRAM 存算单元，这种设计在性能和稳定性上有显著优势。

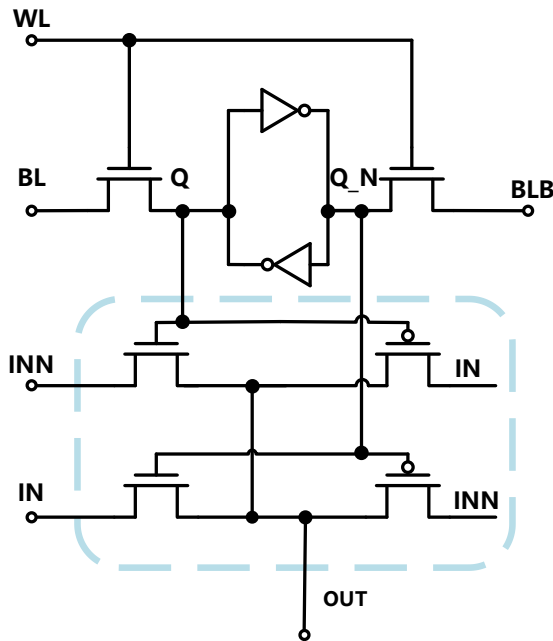


图 3-2 4T 计算单元和 6T SRAM 电路图

3.2.1 6T SRAM 单元的设计

首先，在设计 6T SRAM 单元时，为了确保读写操作的顺利进行，要重点关注其内

部晶体管的尺寸。这种 SRAM 单元通常包括两个上拉管(PMOS)、两个下拉管(NMOS)和两个选通管(NMOS)。在设计时,一个重要的准则是确保这些晶体管的驱动能力呈现出特定的层次结构:下拉管需要有最强的驱动能力,其次是选通管,而上拉管则具有相对较弱的驱动能力。

驱动能力的不同直接影响 SRAM 单元的性能。下拉管需要最强的驱动能力来确保在读取和保持数据时能够快速且有效地充电到节点。选通管,也就是访问晶体管,需要次于下拉管但高于上拉管的驱动能力,以便在读写操作中有效地控制存储节点的电位。最后,上拉管的驱动能力相对较弱,主要是为了在写入'0'时稳定地放电。

除此之外,在 SRAM 设计中,保持交叉耦合反相器之间的平衡对于确保高静态噪声裕度(Static Noise Margin, SNM)和稳定的数据保持能力至关重要。SRAM 单元的 Q 和 QN 线(分别代表输出和反向输出)上的负载对称性在此扮演了一个关键角色。如果这些线上的负载不对称,它可能会导致两个交叉耦合反相器之间的不平衡,从而影响 SRAM 单元的稳定性和静态噪声裕度。

在某些设计中,如论文[23]所述,计算单元仅与 SRAM 单元的 QN 线连接。这种设计导致了反相器之间的不对称性,从而不利于保持数据的稳定性。不对称的负载会改变反相器的电压传递特性(VTC),减小形成在 VTC 曲线上的“正方形”的尺寸,这是衡量噪声裕度的关键参数。结果就是,降低了 SRAM 单元在面对噪声时维持其存储状态的能力。

然而,本文提出的设计方法采取了不同的策略。在这种设计中,计算单元通过 Q 和 QN 线与 SRAM 单元相连接,确保了两线上负载的对称性。这种对称连接有助于维持交叉耦合反相器之间的平衡,从而提高了 SRAM 单元在读取和写入操作中的稳定性。保持 Q 和 QN 线上负载的对称性不仅增强了单元的静态噪声裕度,还提高了整体的数据保持能力和可靠性。

3.2.2 可配置逻辑运算电路的设计

可配置的逻辑运算单元是存算单元中的关键组成部分,其电路图如图 3-3 所示,它由四个 CMOS 管子组成,Q、Q_N 端口分别与 SRAM 的 Q、Q_N 相连,IN、INN 与外部的控制输入模块相连。通过改变 IN、INN 的值,可以完成不同运算。该电路的逻辑表达式为:

$$OUT = Q \cdot INN + Q_N \cdot IN \quad (3-1)$$

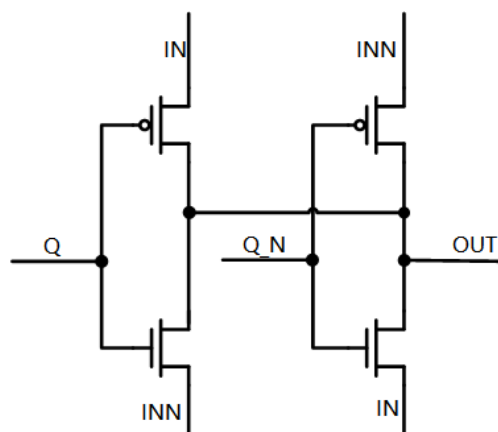


图 3-3 可配置的逻辑运算单元电路图

本结构采用静态逻辑电路设计，相比于论文[19]中提到的基于动态逻辑的读出电路，它有几个显著的优势。

首先，静态逻辑设计的一个主要优点是其相对较低的漏电流。在静态逻辑电路中，由于其稳定的电压状态，漏电流通常比动态逻辑电路中的漏电流小。这是因为动态逻辑电路依赖于电容充放电周期，而在这个过程中容易产生更多的漏电流。因此，采用静态逻辑的电路在长期运行时能够更有效地减少功耗，尤其在低功耗应用中这一点尤为重要。

其次，这种静态逻辑电路能够长时间保持输出数据。这是因为静态逻辑不依赖于电荷存储，而是基于稳定的逻辑门状态，因此能够无限期地保持其输出，直到输入改变。这种特性使得静态逻辑电路在需要长时间保持数据的应用中（如某些类型的存储器和保持电路）更为理想。

此外，这种设计还提高了电路的并行性和吞吐量。在静态逻辑设计中，由于不需要周期性地刷新电荷，可以更容易地实现电路的并行操作。这使得在执行多任务处理或高速运算时，电路能够处理更多的数据，从而提高整体性能。

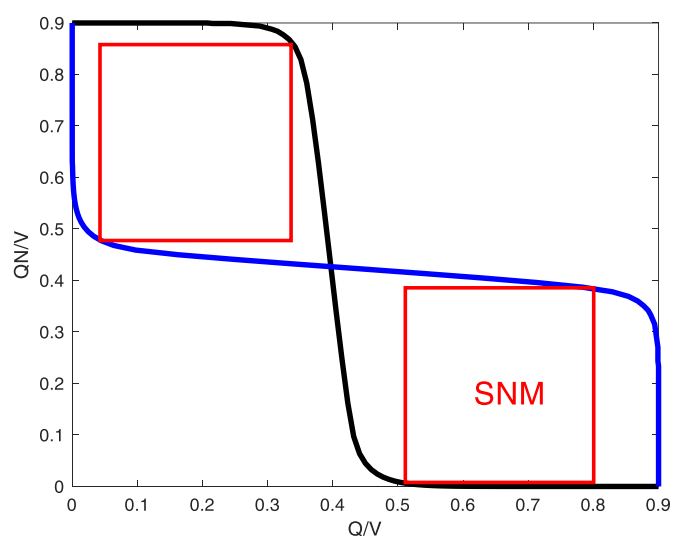
最后，这种设计不需要共享位线（Bit Line, BL）。在动态逻辑设计中，共享位线可能会导致速度下降和信号完整性问题，因为不同的电路单元需要在同一条线上进行通信。本论文提出的 RLOU 电路通过避免这种设计，可以减少这些问题，从而提高信号的可靠性和整体电路的效率。

综上所述，使用本论文提出的电路设计在漏电流、数据保持时间、并行性、吞吐量以及位线共享方面相比于动态逻辑设计具有显著优势，这些特性使得它在许多高性能和低功耗的应用中非常适用。

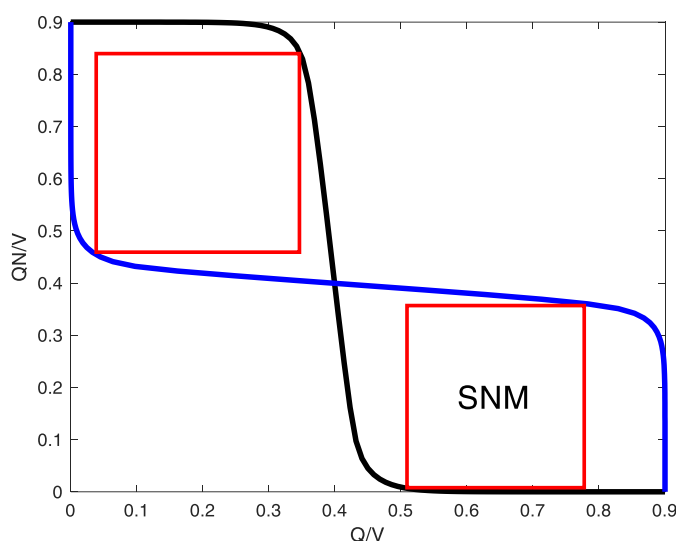
3.2.3 SRAM 存算单元的仿真与分析

本文提出了一种基于 4TRLOU 逻辑计算单元的设计，这种设计与传统使用逻辑门作为计算单元的 SRAM 架构相比，具有显著的优势。特别是在 Q 和 QN 线上，这种设计实现了更加对称的负载分布，这在一定程度上有助于提高存储单元的性能和稳定性。

SRAM 的静态噪声容限 (Static Noise Margin, SNM) 是衡量其抗噪声能力的重要指标，通常由“蝴蝶曲线”图中最大的正方形的边长来表示。静态噪声容限越大，意味着 SRAM 在面对电路噪声时的稳定性和可靠性越强。图 3-4 为使用 RLOU 单元和传统逻辑门单元的 SRAM 性能的仿真图。



(a)



(b)

图 3-4 使用 4TRLOU 单元的 SRAM 蝴蝶曲线图 (b) 使用逻辑门的 SRAM 蝴蝶曲线图

从蝴蝶曲线图中观察到的数据显示,采用 4T RLOU 逻辑计算单元的 SRAM 具有更高的静态噪声容限——约 300mV,相比之下,使用传统逻辑门作为计算单元的 SRAM 的 SNM 约为 280mV。这一仿真结果与理论分析相符合,表明 4TRLOU 单元设计在提高 SRAM 的噪声抵抗能力方面具有明显的优势。这种提高的噪声容限不仅增强了 SRAM 单元的稳定性,也有助于提高整体存储系统的可靠性。

使用 4T RLOU 单元的 SRAM 版图如图 3-5 所示, MOS 管的宽长比均遵循 3.2 节的理论分析,这有助于 SRAM 顺利完成读写操作。这里将 SRAM 的宽设为 1 μm ,这有助于与后面的加法器树版图相对应,其面积为 1 $\mu\text{m} \times 1.17\mu\text{m}$ 。

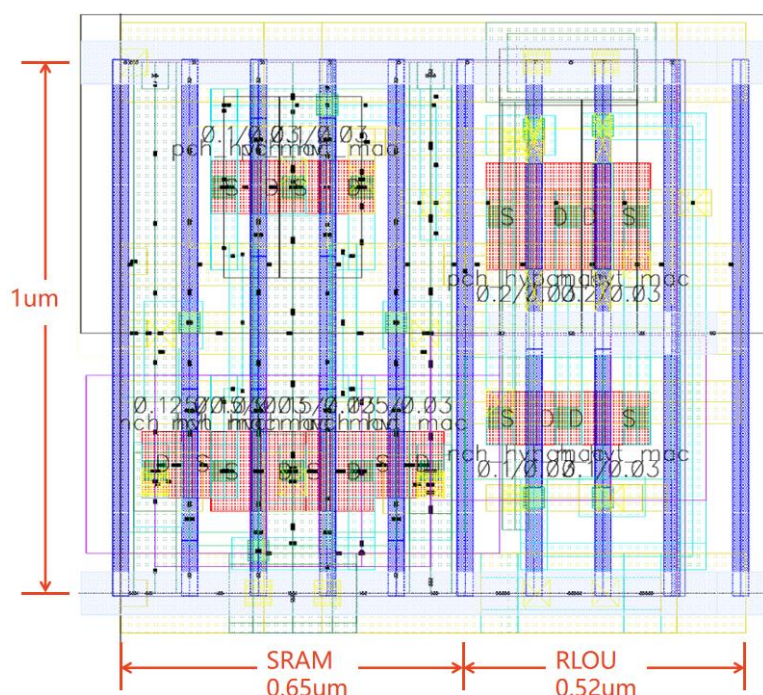


图 3-5 使用 4T RLOU 单元的 SRAM 版图

3.2.3.1 SRAM 的保持噪声容限

接下来对 4T RLOU 逻辑计算单元的 SRAM 进行噪声容限仿真。首先是保持噪声容限 (Hold Static Noise Margin, HSNM), HSNM 用于评估 SRAM 在存储数据时的稳定性。仿真电路的搭建遵循了论文[35]中提出的方法,如图 3-6 所示。BL、BLB 保持高电平, WL 关闭, IN 输入为 0, 在两个耦合的反相器间引入一个斜波电压源 V_a , 该电压源用于模拟由 PVT 产生的失调电压, V_a 从 0 逐渐增加到电源电压 VDD, 当存储节点翻转时, V_a 的值即为 HSNM。

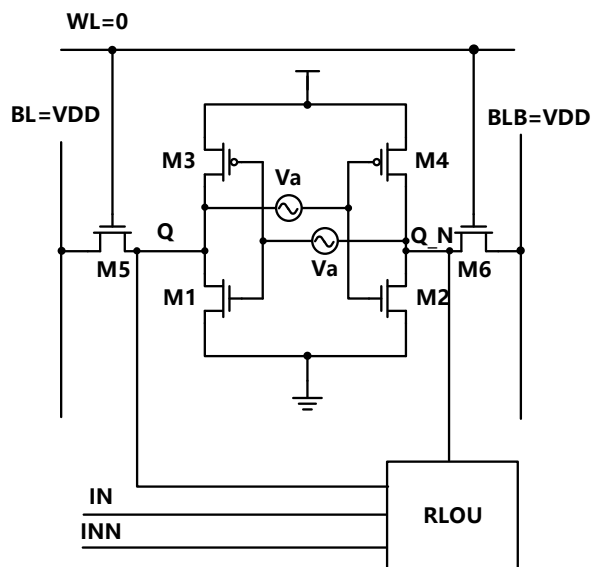


图 3-6 HSNM 测试电路

根据上述测试方法，在 0.9V 的电压下对 SRAM 单元进行 HSNM 仿真。在进行 500 次蒙特卡洛仿真后，结果如图 3-7 所示。本设计提出的 SRAM 存算单元的 HSNM 为 345mV，可以满足 SRAM 的稳定性需求^[35]。

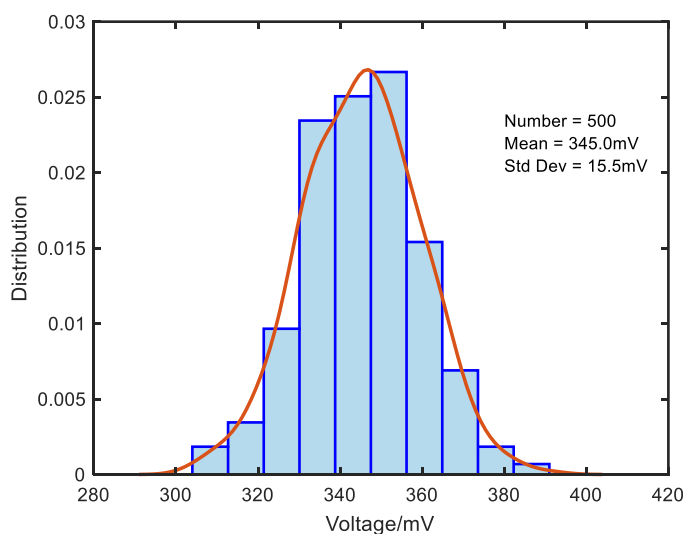


图 3-7 HSNM 仿真结果

3.2.3.2 SRAM 的读噪声容限

读噪声容限（Read Static Noise Margin）是用于评估 SRAM 单元在读数据时的稳定性。搭建的仿真电路如所示，RSNM 的测试方法与 HSNM 一致，不同之处是打开了 WL 线，模拟 SRAM 单元在读状态时的条件。这里同样进行了 500 次蒙特卡洛仿真，测试电路与测试结果如图 3-8 所示。根据仿真结果可以看出，SRAM 计算单元的 RSNM 为 140.9mV，满足 SRAM 单元对 RSNM 的需求。

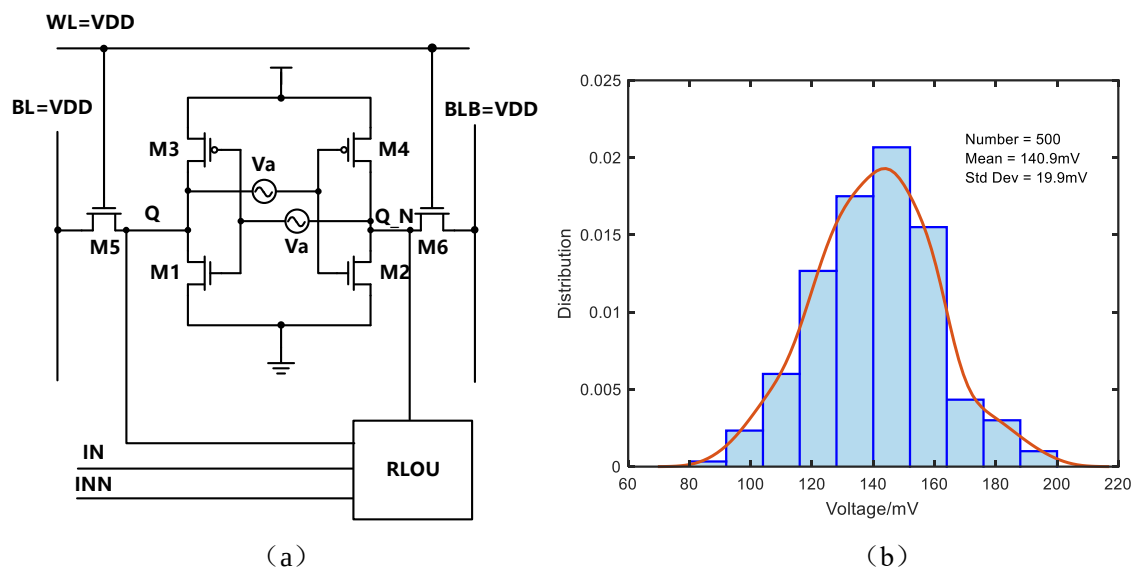


图 3-8 (a) RSNM 的测试电路 (b) RSNM 的蒙特卡洛仿真结果

3.2.3.3 SRAM 的写容限

除了 SNM, 写容限 (Write Margin) 也是衡量 SRAM 稳定性的一个重要指标。WM 主要用于评估 SRAM 单元数据的写入能力, WM 越大, 表示 SRAM 越容易被写入。这里可以使用字线电压扫描法测量 WM^[36]。WM 的测试电路如图 3-9 所示, 具体的测试步骤如下: 首先将 BL 线置为高电平, BLB 连接到低电平, 同时将 WL 与一个斜波电压源相连。接着初始化存储节点, 将 Q 的初始值设为“0”, 反向节点 Q_N 初始值设为“1”, 用于模拟写入新数据的初始状态。最后扫描 WL 的值, 在 WL 从 0 变化到 VDD 的过程中, 存储节点翻转的那一刻, 标志着数据被成功写入。WM 的值则是通过从电源电压 VDD 中减去这个字线电压来计算得出。

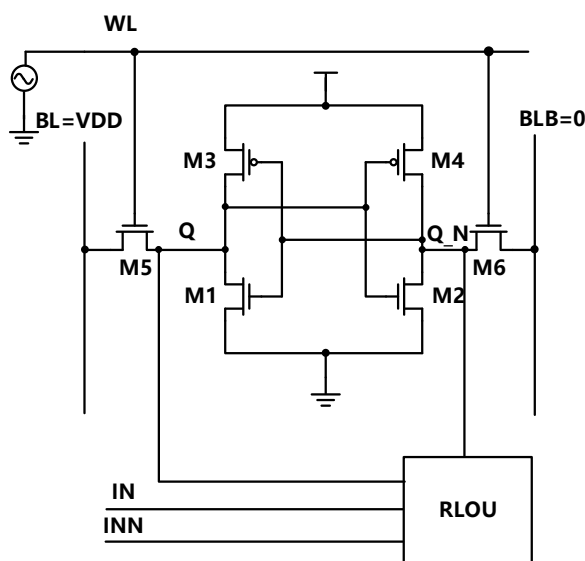


图 3-9 WM 测试电路图

WM 的测试结果如图 3-10 所示,由于本设计中的 SRAM 单元为对称结构,写“0”与写“1”的情况是一样的,因此只需要进行一次测试。根据仿真图可以看出,SRAM 单元的 WM 为 261.6mV,满足 SRAM 写容限的设计需求^[36]。

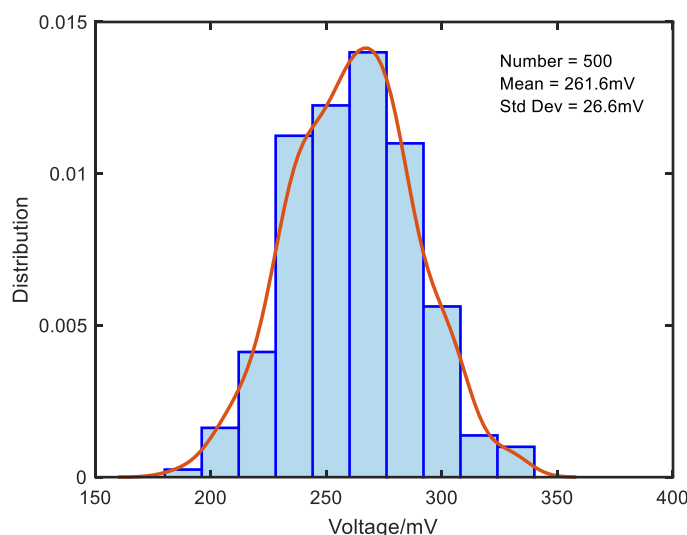


图 3-10 WM 蒙特卡洛仿真结果

3.3 加法器树方案比较

为了选择最适合存算架构的加法树,以实现最高的能效比,可以考虑降低电路的功耗和延时。本节提出了三种不同加法器树的设计方案,详细分析并比较了它们的性能指标,包括优缺点。这对于后续提出的存算一体架构设计提供了重要的参考依据,有助于优化整体的能效比和性能。

3.3.1 全加器电路

本节主要介绍全加器及加法器电路的基本理论知识,并对加法器树的延时进行分析,为后续章节的加法器树延时及功耗计算作参考。

3.3.1.1 全加器基本结构

全加器是复杂算术电路的核心元件,也常常是限制速度的关键器件,因此仔细优化全加器极其重要。静态 CMOS 全加器可被分为两大类:互补 CMOS 型和传输门型。

表 3-1 列出了一个二进制全加器的真值表,A、B 是加法器的输入, C_i 是进位输入,S 是和输出,而 C_o 是进位输出,S 和 C_o 的表达式如下:

$$S = A \oplus B \oplus C \quad (3-2)$$

$$C_o = AB + BC_i + AC_i = AB + (A \oplus B) C_i \quad (3-3)$$

根据真值表,可以将产生 S 和 C_o 的中间信号 G (进位产生, generate)、D (进位取消, delete) 和 P (进位传播, propagate) 用函数定义出来,其公式如下:

$$G = AB \quad (3-4)$$

$$D = \bar{A}\bar{B} \quad (3-5)$$

$$P = A \oplus B \quad (3-6)$$

可以把 S 和 C_o 写为 P 和 G 的函数:

$$C_o(G, P) = G + PC_i \quad (3-7)$$

$$S(G, P) = P \oplus C_i \quad (3-8)$$

表 3-1 全加器真值表

A	B	C_i	S	C_o	进位状态
0	0	0	0	0	取消
0	0	1	1	0	取消
0	1	0	1	0	传播
0	1	1	0	1	传播
1	0	0	1	0	传播
1	0	1	0	1	传播
1	1	0	0	1	产生/传播
1	1	1	1	1	产生/传播

图 3-11 是一个互补 CMOS 型镜像加法器^[37],它的工作基础是式 (3-7) 与 (3-8),其中需要注意的是连接 C_i 的晶体管在最近进门的输出端处,这是为了使进位传播尽量快,减少加法器的延时。同时,需要优化进位电路中的晶体管尺寸,使节点 C_o 出的电容最小以改善进位传播的速度。

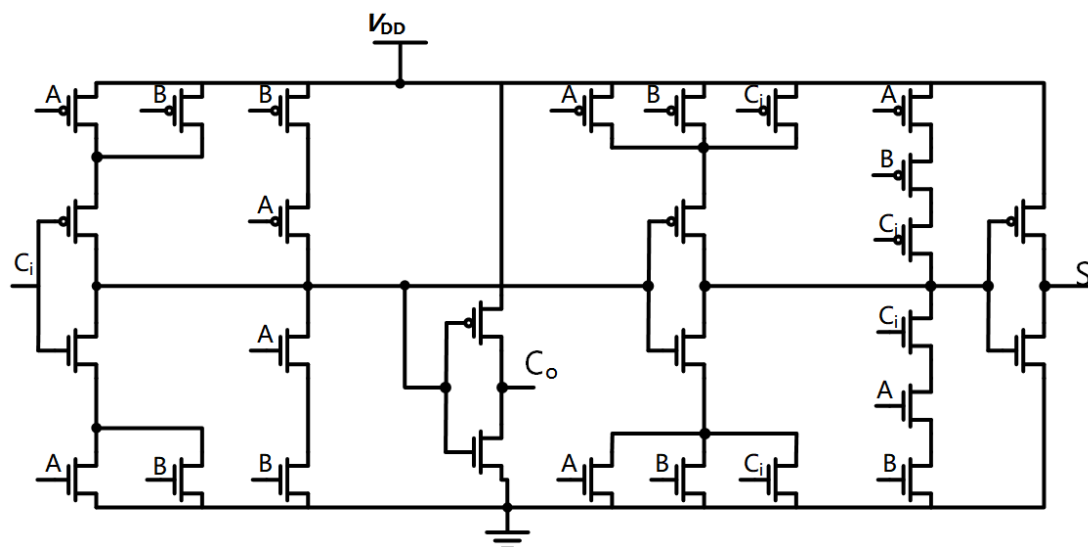


图 3-11 镜像加法器电路图

这种互补式加法器的优势在于，它对于电压的缩放和晶体管尺寸调整具有强大的鲁棒性，这意味着元件在低电压或者任意晶体管尺寸下工作时可靠性强。此外，由于互补 CMOS 具有较少的互连导线，在版图设计时电路更易于实现。

图 3-12 是基于传输门实现的全加器^[23]，一共使用了 14 个 MOS 管。传输门逻辑电路是一种特殊的电路^[37,38]，它由一个 PMOS 和一个 NMOS 并联而成，这两个晶体管由互补的控制信号控制，当它们同时开启时，可以无压降地传递“0”和“1”。在加法器电路中，通过这种可提供全摆幅的器件可以设计出晶体管数量更少的加法器电路，降低功耗。相比于互补型加法器，传输门型加法器的缺点主要是，工作电压范围小且延迟高。

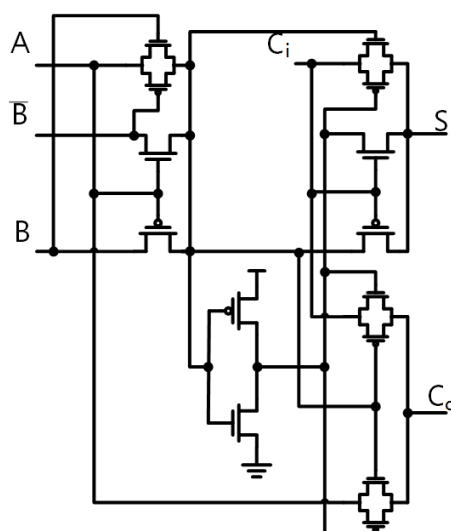


图 3-12 14T 的传输门型加法器

在传输门型加法器中，这种 14T 全加器，与使用较少晶体管数量的如 11T^[39]或 10T^[40]

全加器相比,具有明显的优势。它能提供全摆幅输出,意味着在电压传递过程中不会发生电压损失。因此,14T 全加器相较于其他类型的全加器,展现出更强的驱动能力。这些特点使得 14T 全加器在保持稳定输出和处理高负载方面表现更佳。

3.3.1.2 多位相加的加法器树结构

加法器根据应用在计算机运算中可被分为两大类:一类是涉及链状结构的应用^[41,42],比如串行进位加法器(Ripple Carry Adders, RCA)和阵列乘法器,这类加法器的关键路径通常是进位信号,这就要求全加器快速生成进位输出信号。另一类是涉及树状结构的应用,比如 Wallace-Dadda 树乘法器和数字滤波器^[42,43,44],全加器的作用是构建一个多层的树结构,将部分积压缩为一个保存进位的数字,最后通过进位传播加法器将其转换为正常的二进制数。高比特数加法中,相比于链状结构,树状结构的算法更快,但是大型树状结构加法器电路的实现结构不规则、布线复杂,会浪费芯片面积。因此,加法器树的设计方案需要根据各种因素决定。

链状结构的加法器中,目前应用较广的是串行进位加法器^[45],串行进位加法器又被称为“行波进位加法器”,这是最简单最基本的加法器结构。串行进位加法器一次只能进行一位运算,进位依次通过每一比特位,在高比特数相加时速度很慢。

在树状结构的加法器中,华莱士(Wallace)树乘法器便是一个具有代表性的例子。Wallace 树采用“并行”的思想,改变全加器的使用方式,将其作为压缩器,消除进位链。Wallace 树的优点是每层只有 $O(1)$ 的传播延时,其延迟时间随位数对数增加,这将有助于主要速度增益。

3.3.2 28T 与 16T 交替的加法器树

根据以上章节所述,在存算一体架构中,一个周期会产生 256 个 4bit 的中间积,而加法器树需要将这些数据相加起来。论文[23]中设计的加法器树如图 3-13 所示,这种加法器树以二叉树的方式连接,交替使用 28T 与 16T 的全加器,其目的是降低功耗与面积,提高加法器树的面积效益和能效比。

其中,16T 的加法器由图 3-12 所示的 14T 加法器改进而来,图 3-12 中输入信号 B 需要进行反相输入,因此在 14T 的基础上再加一个反相器形成 16T 的全加器,这样才能与 28T 的全加器连接形成行波进位加法器。

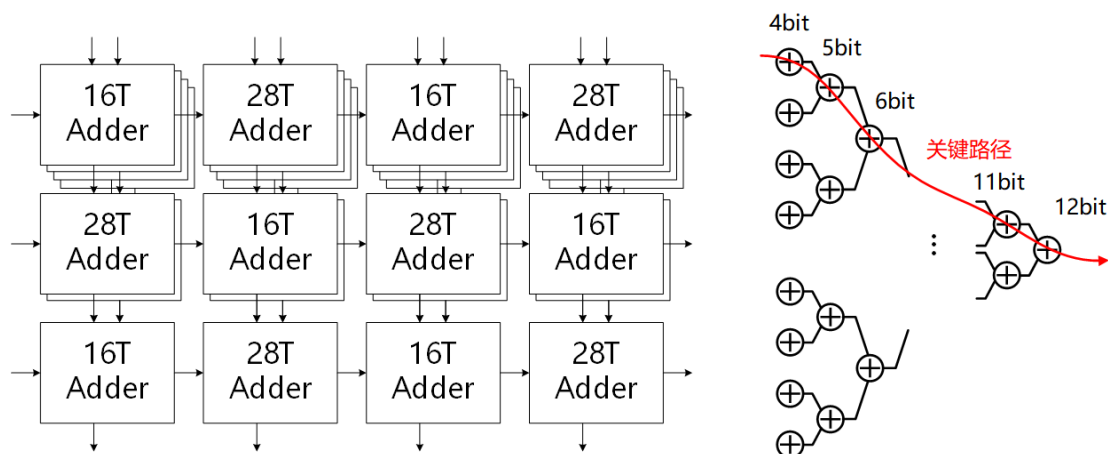


图 3-13 28T 与 16T 混合的逐级进位加法器树

在卷积神经网络中，为了同时支持有符号数和无符号数的运算，数据相加的过程中需要添加一个符号扩展电路，用于处理最高位的符号位。符号扩展电路的真值表如表 3-2 所示，在有符号数加法时，不存在最高位都是 0 而有进位和最高位都是 1 而没有进位的情况。根据真值表，可以推出符号扩展电路的结构图，如图 3-14(a)所示。

图 3-14(c)中，A、B 为加数和被加数的最高位，进位输入 C_i 与前一位计算结果的进位输出 C_o 相连，这样就可以把 N 个 1 位的全加器 (Full Adder, FA) 电路串联起来，构成一个 N 位的加法器，为了节省面积，行波进位加法器的最低位加法使用半加器 (Half Adder, HA) 完成，其电路图如图 3-14(b)所示。通过该电路的延时取决于传播必须通过的逻辑级的数目，以及所加的输入信号。在本结构中，进位必须通过最低有效位 (Least Significant Bit, LSB) 一直波动传到最高有效位 (Most Significant Bit, MSB)。最坏情况的延时 (关键路径) 正比于输入字的位数 N 并近似为：

$$t_{adder} = (N - 1)t_{carry} + t_{sum} \quad (3-9)$$

其中， t_{carry} 和 t_{sum} 分别等于 C_i 至 C_o 及 S 的传播延时。要计算 256 组 4bit 数据的和则需要 8 级加法器，不考虑符号扩展电路的延时，总的传播时延为 $26t_{carry-28T} + 26t_{carry-16T} + 8t_{sum}$ 。

表 3-2 行波进位加法器符号扩展电路的真值表

A	B	C_i	S(SIGN = 0)	S(SIGN = 1)
0	0	0	0	0
0	0	1	1	不存在
0	1	0	0	1
0	1	1	1	0

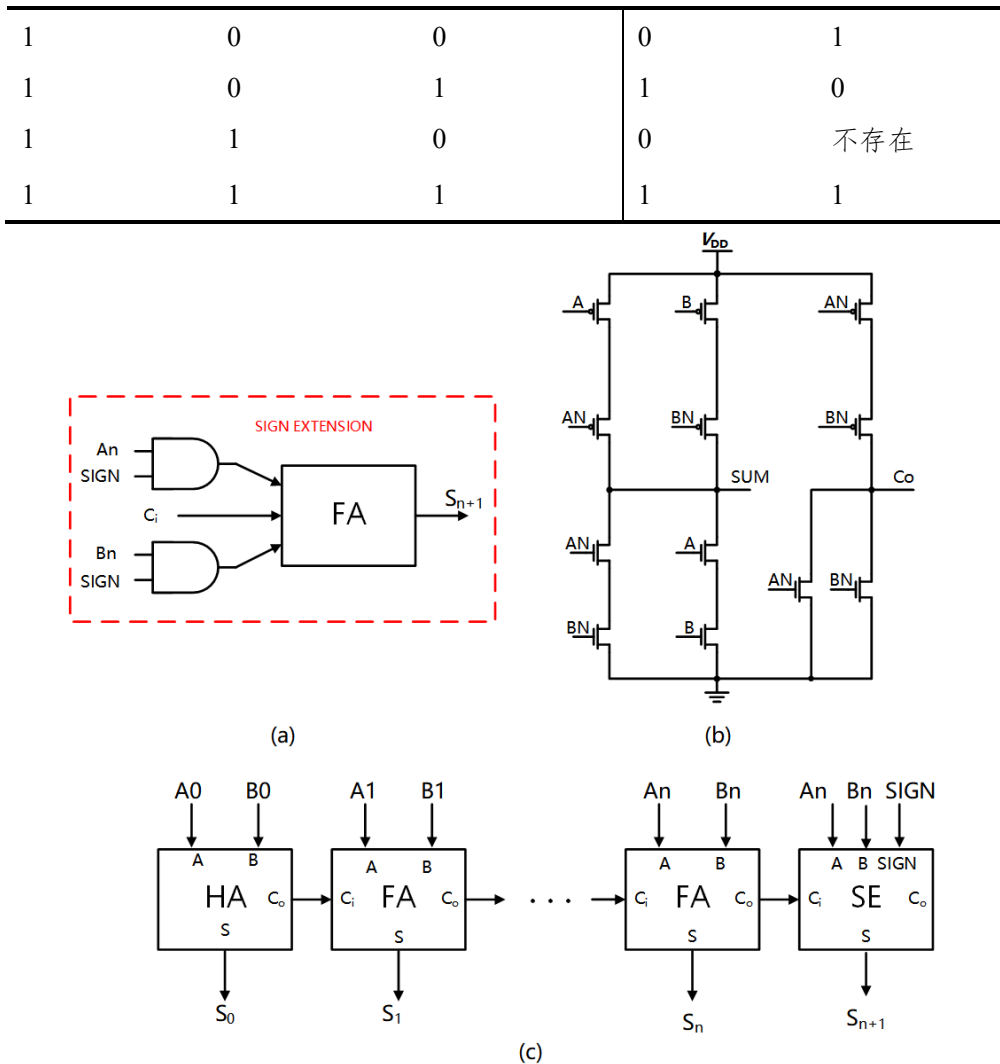


图 3-14 (a)符号扩展电路 (b) 半加器电路图 (c) 行波进位加法器结构

3.3.3 全 16T 的加法器树

在存算一体结构中，加法器树通常会占用大量的面积和产生较大的功耗，与上一节中提到的基于 28T 的加法器树不同，本节主要使用了一个简化的基于 16T 的传输门型加法器单元来构建加法器树，其电路图如图 3-12 所示。16T 的传输门型加法器主要通过传输门传递信号，而传输门本身没有驱动能力，需要靠前一级的驱动能力，因此这种结构存在驱动能力不足的问题。仅使用 16T 的加法器单元可能会导致计算结果出现错误，并且运算速度过慢，使存算芯片在一个周期内无法完成运算。

为了解决以上问题，本文在加法器树路径中分配了缓冲器，但是缓冲器不是在每一级上加的，具体级数由仿真结果确定（详见 3.3.6.3 节），基于 16T 的加法器树结构图如所示。

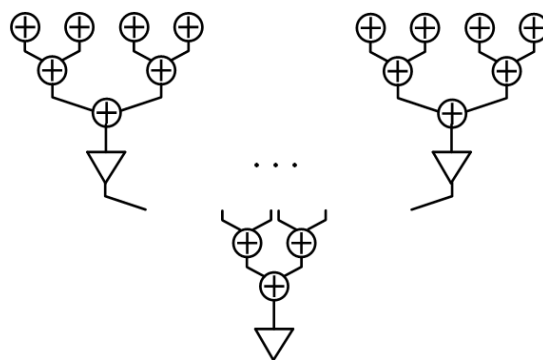


图 3-15 基于 16T 的加法器树结构图

这种设计明显减少了晶体管的数量，且有效地降低了硬件的复杂性，基于 16T 的加法器树一级的传播延时与式 (3-9) 一致，则其总传播延时约为 $52t_{\text{carry-16T}} + 8t_{\text{sum}} + n t_{\text{buffer}}$ ，其中 n 指缓冲器的级数。从延时表达式中可以看出，16T 加法树的速度相比于交替结构的加法树有所降低。

3.3.4 混合华莱士树结构的加法器树

为了提高加法器树的能效比，除了降低功耗外，缩短延时也是关键。随着比特数的增加，链状结构的行波进位加法器的传播延时会随着位数增大而线性增大，在设计一个快速加法器时，优化 t_{carry} 比优化 t_{sum} 重要得多，因为后者对 t_{adder} 总延时影响较小。因此，本设计采用了华莱士树结构的加法器，这是一种树状结构。华莱士树为了优化延时，改变了全加器的连接方式，将其作为 3-2 压缩器使用，3-2 压缩器也被称为进位保留加法器 (Carry Save Adder, CSA)，它的输入为三个加数，输出为和与进位信号，进位信号直接与下一级压缩器相连，取消了串行进位的情况，减少时延。

由于存算阵列输出的是 256 组 4bit 的数据，若直接使用华莱士树对数据进行压缩，其压缩率与面积效益会较低，因此，本文提出了一种混合华莱士树的加法器结构。也就是在华莱士树结构中，前几级使用传统的以二叉树拓扑连接的行波进位加法器，当位数增加时再使用华莱士树对加法运算进行加速，这样就结合了两类加法器的优点，优化加法器树的性能指标。

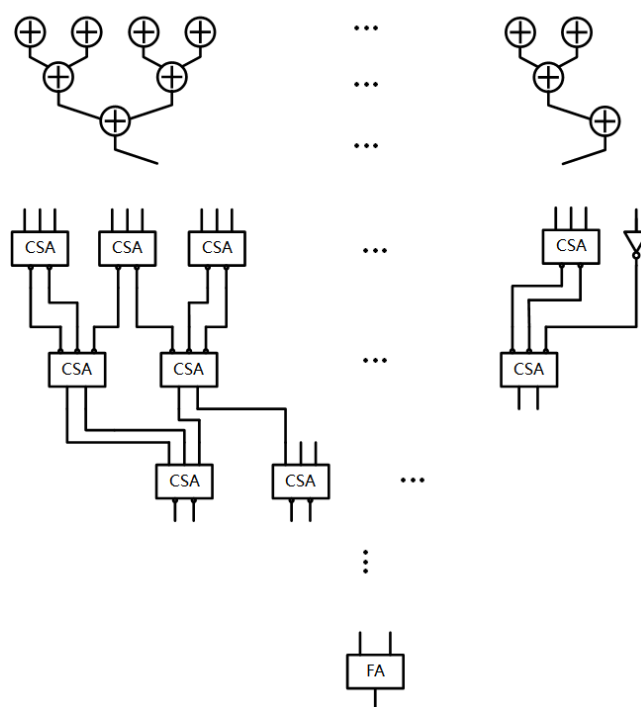


图 3-16 混合华莱士树加法器结构图

图 3-16 展示了混合华莱士树加法器的示意图，其中使用的 CSA 为经过修改的加法器，CSA 优化了图 3-11 中镜像加法器输出端的两个反相器，将 28T 的全加器改为 24T 的全加器。这利用了全加器的一个逻辑性质：把一个全加器的所有输入反相，它的所有输入也反相，这种性质可以用以下两个式子描述：

$$\bar{S}(A, B, C_i) = S(\bar{A}, \bar{B}, \bar{C}_i) \quad (3-10)$$

$$\bar{C}_o(A, B, C_i) = C_o(\bar{A}, \bar{B}, \bar{C}_i) \quad (3-11)$$

这就是全加器的反相特性，这种设计方式不仅减少了反相器的数量，还改善了加法器树的运算速度。

由于优化了全加器的反相器，在搭建混合华莱士树的过程中，要注意相邻两级加法器的输入是反相的，华莱士树以 3-2 的比例压缩，因此在某些级数中有部分积无法加入下一级，需要其保留至下两级再输入，并且要对部分积进行反相逻辑运算，才能保证结果的正确性。除此之外，构建华莱士树时要注意“权重对齐”，全加器的进位输出比和输出高一位，因此在下一级加法运算时要对和的高位进行符号拓展，符号拓展电路的真值表如表 3-3 所示。根据真值表可推出符号扩展电路的逻辑表达式：

$$S_{n+1} = S_n \cdot SIGN \quad (3-12)$$

$$\overline{S_{n+1}} = \overline{\overline{S_n} \cdot SIGN} \quad (3-13)$$

华莱士树的符号扩展电路图如图 3-17 所示, 结构的选择取决于加法器输出的相位。相比于行波进位加法器, 华莱士树的符号扩展电路面积小且更易于实现。

表 3-3 华莱士树的符号扩展电路真值表

SIGN	S_n	S_{n+1}	SIGN	$\overline{S_n}$	$\overline{S_{n+1}}$
0	0	0	0	0	1
0	1	0	0	1	1
1	0	0	1	0	0
1	1	1	1	1	1

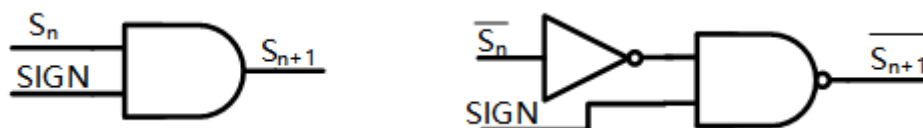


图 3-17 华莱士树的符号扩展电路

由于华莱士树适合高比特数加法器, 一般在 $N > 6$ 时使用, 因此混合结构会在前几级和最后一级使用行波进位加法器, 中间部分使用华莱士树结构, 以 256 组 4bit 输入数据为例, 其传播时延约为:

$$t_{Wallace} = \left[\frac{(2+x)x}{2} + 11 \right] t_{carry} + (x+1)t_{sum} \quad (3-14)$$

其中, x 为行波进位加法器的级数, y 为华莱士树加法器的级数。在混合结构的加法器中, 延时的大小主要由进位时间决定。

3.3.5 加法器树的仿真分析

本节主要对上文提出加法器树架构进行仿真分析, 仿真条件均为 28nm 工艺, 0.9V 工作电压和 27°C 的温度, 其中输入为 256 组 4 比特随机数据。

3.3.5.1 28T 与 16T 全加器的仿真与分析

在 28T 全加器单元中, 进位电路是影响延时的重要因素, 因此在设计时, 需要优化进位电路的尺寸, 使其逻辑努力为 2, 并且将进位级的尺寸扩大到求和级尺寸的 3-4 倍, 以达到最小延时^[37]。16T 的全加器是使用进位传播-产生模型设计的, 这种加法器有一个特点就是它的和与进位输出具有相近的延时。图 3-18 为 28nm 工艺下 28T 全加器和 16T 全加器的版图。

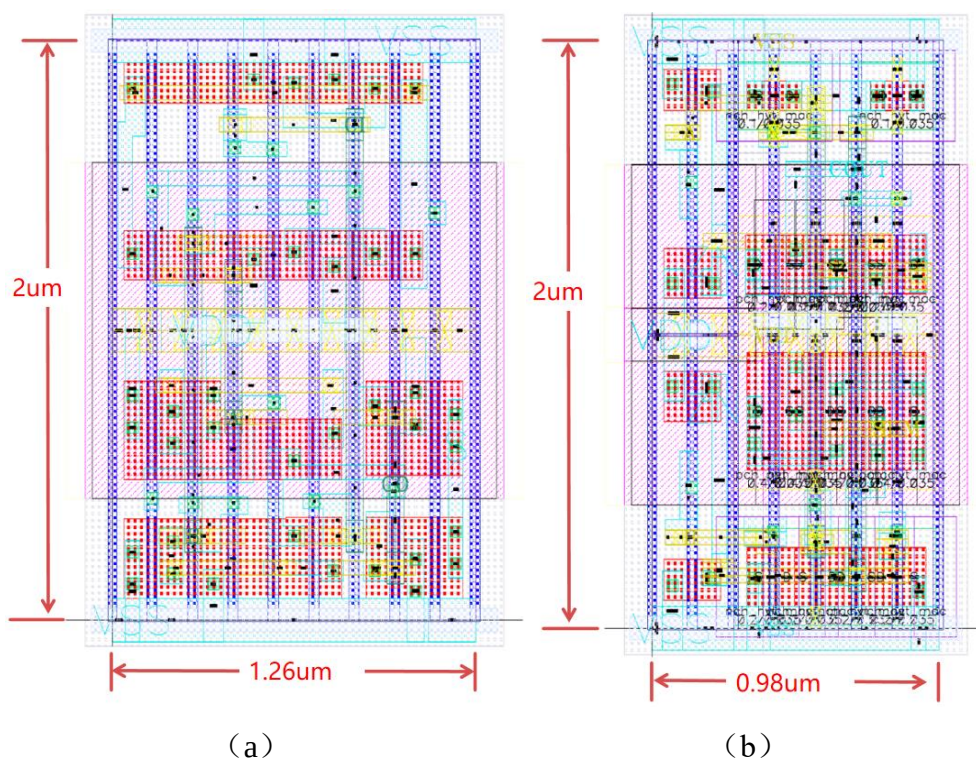


图 3-18 (a) 28 管全加器单元版图 (b) 16 管全加器单元版图

在设计中，28T 全加器遵循了标准单元库的布局，而 16T 全加器因包含传输门结构，布局参考了部分标准单元。两种加法器设计有相同的高度，以便于后续布线连接。28T 全加器的面积约为 $2.52\mu\text{m}^2$ ，16T 全加器的面积约为 $1.96\mu\text{m}^2$ ，后者面积大约为前者的 77.8%。

本文使用 Cadence 仿真软件，在 0.9V 的工作电压，27℃ 的仿真条件下，分别对 28T 和 16T 全加器的延时及功耗进行了仿真。其中时钟周期为 500MHz，仿真包括了所有的输入模式，即输入数据 A，B， C_i 为 ‘000’ ~ ‘111’，在每次输入时，初始输入为 ‘000’。激励模块由 Verilog 代码生成，将激励模块与全加器模块相连，使用 ams 仿真器对混合信号电路进行仿真。首先是功耗仿真，其仿真结果如图 3-19 所示。

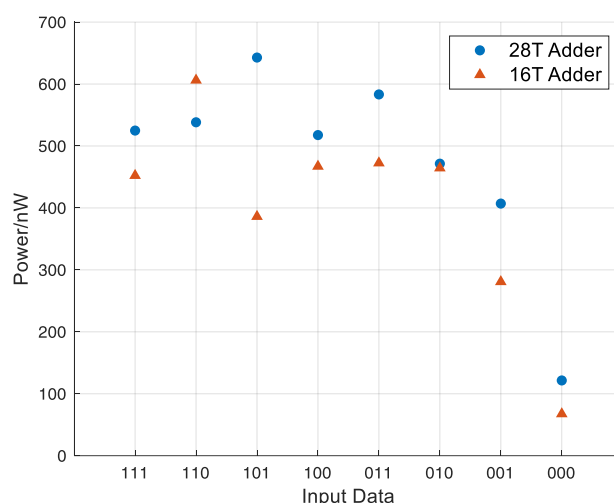


图 3-19 全加器所有输入模式的功耗仿真

从图中可以看出,在相同的仿真条件下,16T 和 28T 全加器的延迟都表现出显著的波动。总体来看,16T 全加器由于晶体管数量较少,因此功耗较低。然而,在镜像加法器中,由于充电和放电路径上的负载电容不同,导致不同输入条件下延迟的变化较大。在 500MHz 的工作频率和 0.9V 的工作电压下,28T 全加器的平均功耗约为 475.8nW,16T 全加器的平均功耗约为 399.6nW。

图 3-20 展示了不同输入数据条件下两种全加器的延时仿真结果。结果显示,两种全加器的延时都存在浮动。在没有进位的情况下,16T 全加器的延时较低,这得益于其传输门结构和简化的逻辑设计。根据计算,28T 全加器的平均延时为 19.8ps,16T 全加器平均延时约为 9.5ps。

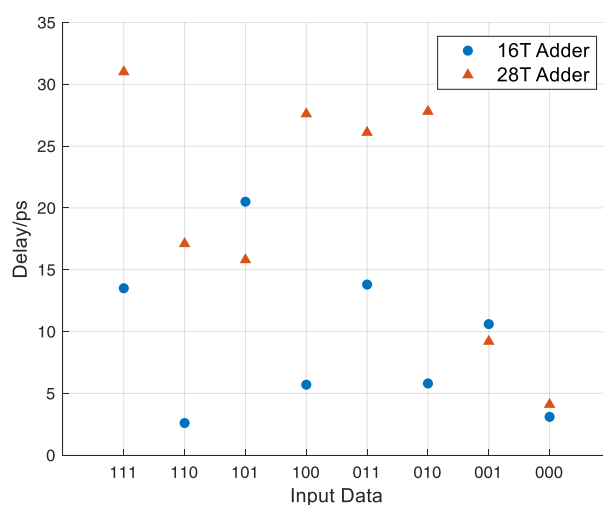


图 3-20 全加器所有输入模式的延时仿真

根据以上仿真结果,对 16T 全加器和 28T 全加器性能指标进行归一化,结果如表

3-4 所示。从表中可以看出，在不存在进位链的情况下，16T 全加器在延时和功耗上具有较大优势，这是因为 16T 全加器使用了传输门来实现 XOR 运算逻辑，使之具有延迟低、功耗低、面积小的优势。

但是，在评估 16T 和 28T 全加器时，除了延时、功耗和面积的对比之外，电路的稳定性和驱动能力也是重要的考量因素。稳定性涉及到电路在各种电压和环境下的可靠性，这决定了全加器在实际操作中的健壮性。驱动能力指的是全加器输出端对后级电路的驱动效能，这在高负载或长线路的情况下尤为重要。16T 全加器虽然没有进位链的情况下在延时和功耗上表现出优势，但在实际应用中，必须确保电路能够稳定运作且具备足够的驱动能力以适应不同的负载条件。28T 的全加器由于具有静态 CMOS 的互补特性，在稳定性方面表现更佳，适用于高性能运算电路。在设计选择时，必须综合考虑这些因素，确保电路的全面性与适应性。

表 3-4 16T 与 28T 全加器延时与功耗对比

	延时	功耗	面积
28T 全加器	1	1	1
16T 全加器	0.48	0.87	0.78

3.3.5.2 28T 与 16T 混合的加法器树仿真与分析

根据以上结论，采用 16T 全加器替换部分 28T 全加器的策略可以有效降低加法器树的计算功耗并减少其总体面积。同时，结合 28T 和 16T 全加器的架构能够确保每级加法器之间具有足够的驱动能力，平衡了性能和稳定性的需求。这种混合使用的策略能够充分利用两种全加器的优势，提高整体电路的效率和可靠性。

根据 3.3.2 节所述搭建加法器树电路，版图如图 3-21 所示，256 组 4 比特输入的 28T 与 16T 混合的加法器树面积约为 5248um²。

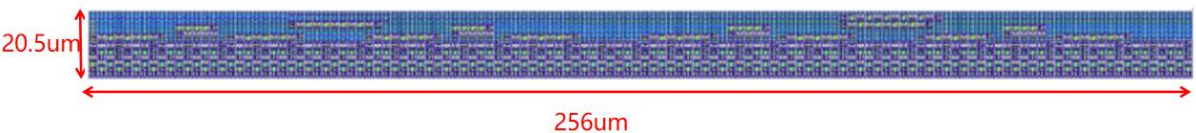


图 3-21 28T 与 16T 混合的加法器树版图

如图 3-22 所示，通过对加法器树的后端仿真，分析了不同电源电压下的延时和功耗。在仿真时，工作电压扫描范围为 0.8 至 1.0V。输入数据为 256 组 4 比特随机数据，通过 Verilog 代码生成。仿真结果表明同时，延时随着电压升高而降低，功耗随着电压的

升高而增加,这符合常理,因为更高的电压通常意味着更快的运算速度和更高的能耗。经计算,在 0.9V 电压下,加法器树的能效比达到最高。这意味着,在 0.9V 时,加法器树的运算性能最优化,是功率与性能的最佳平衡点。

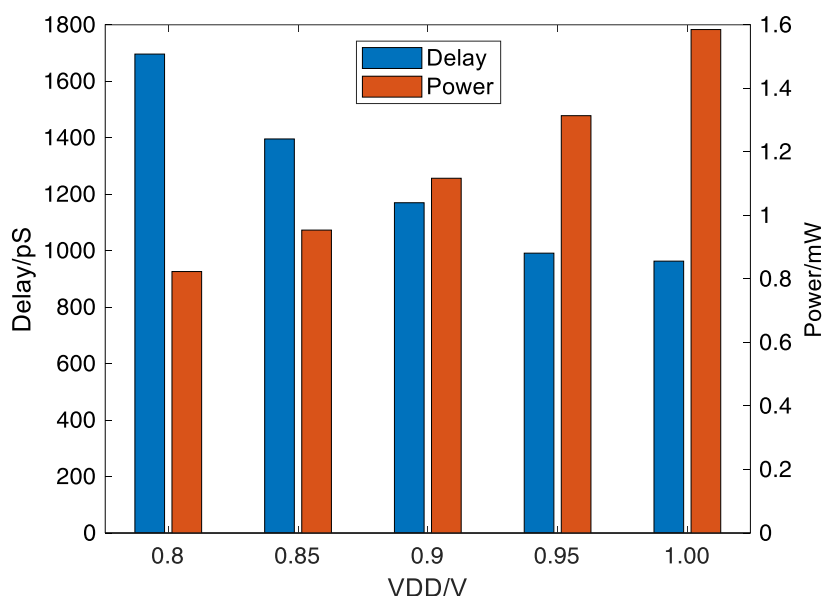


图 3-22 28T 与 16T 交替的加法器树延时与功耗仿真结果

3.3.5.3 全 16T 的加法器树仿真结果

交替使用 28T 和 16T 全加器构成的加法器树是在功耗和面积之间进行权衡的一种方案。在存算一体架构中,加法器的功耗对总体性能有着显著影响。鉴于时钟存在,只要在一个时钟周期内完成计算,就有可能采用功耗更低的设计。因此,提出了全 16T 加法器的方案,可以进一步降低电路的功耗,提升能效比。

在 3.3.3 节中提到,由于 16T 全加器的驱动能力不足,在加法器树设计中需要增加缓冲器来增强信号。为了确定所需的缓冲器级数,首先需测试 16T 全加器在不同工作电压下进行准确计算的最低输入电压。图 3-23 的仿真结果显示,当工作电压为 0.9V 时,16T 全加器的最低输入电压是 480mV,而输出为全摆幅输出。这意味着,在加法器树的树形连接中,缓冲器的主要作用是解决延时问题,而不是电压损失。在选择缓冲器级数时,应重点考虑如何减少延时,以确保计算结果的时效性和准确性。

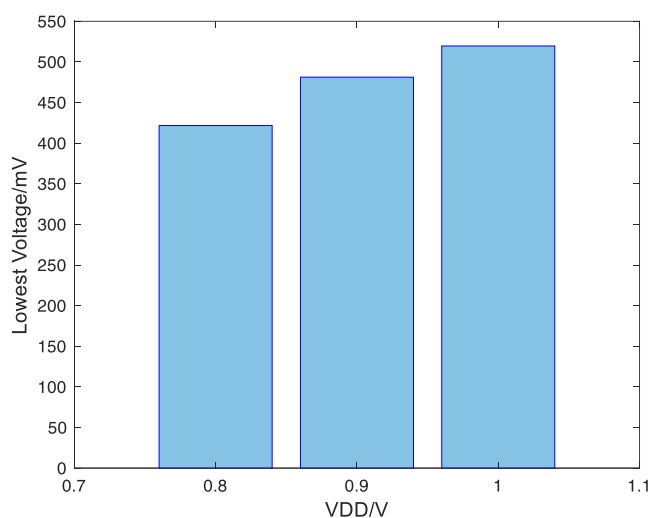


图 3-23 16T 全加器最低输入电压仿真

接下来对 16T 加法器树进行延时仿真，并与方案一的加法器树进行对比，仿真结果如图 3-24 所示。这幅图显示了 16T 全加器树和 28T 全加器树在不同级数的延时仿真结果。从图中可以看出，随着加法器树级数的增加，两种方案的延时差异变得越来越大。特别是在第三级之后，16T 全加器树的延时急剧上升，这可能是由于其驱动能力不足在树形结构中被放大，导致整体延时显著增加。这个结果突显了在设计 16T 加法器树时需要特别注意信号驱动和缓冲问题。

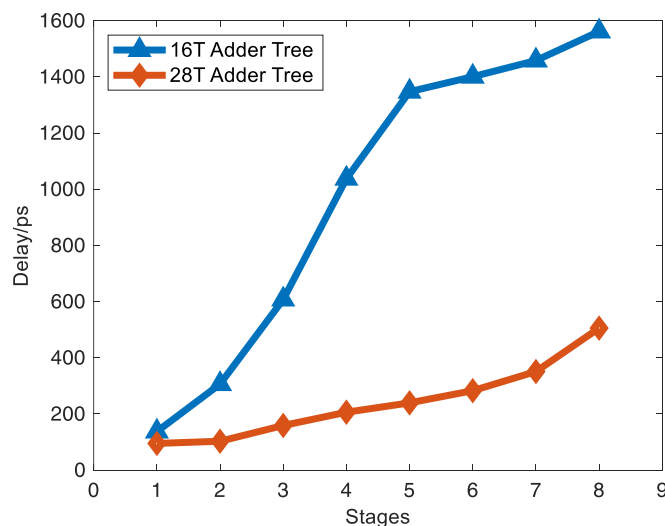


图 3-24 16T 加法器树与方案一加法器树延时仿真

根据以上的仿真结果分析，本文提出了一种全 16T 加法器树的新设计，该设计采用了三级缓冲器技术。具体来说，是在每三级加法器后增加了一组缓冲器，这样做的目的是在确保加法器树满足速度要求的同时，尽可能地限制面积的增加。新加法器架构

的仿真结果如图 3-25 所示, 我们可以观察到, 与未添加缓冲器的加法器树相比, 新设计的延迟显著降低。新架构的最大延时大约是方案一延时的 125%, 但延时的增加仍在可接受范围内。

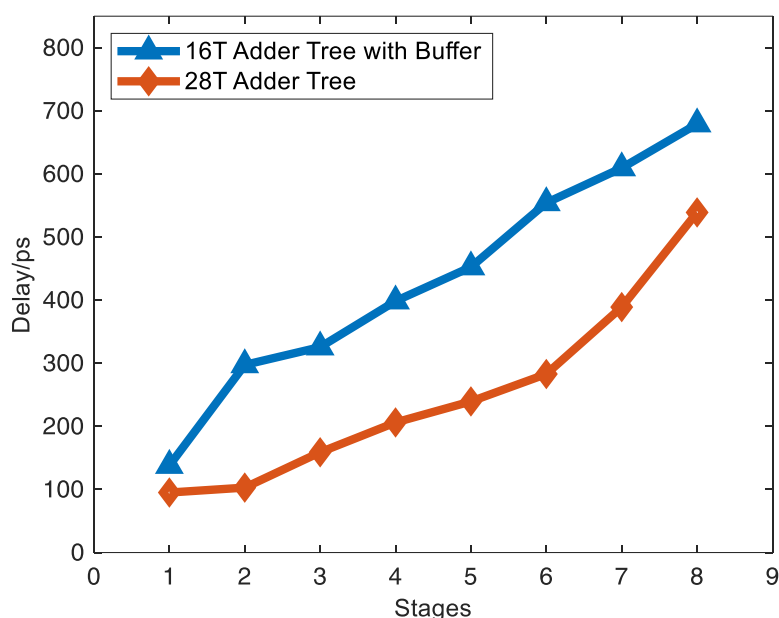


图 3-25 带缓冲器的 16T 加法器树延时仿真结果

虽然加入缓冲器的设计在延迟上有显著改善, 但是还需要评估缓冲器对面积和功耗的影响。方案二的新架构版图如图 3-26 所示, 其长宽为 $17.8\mu\text{m} \times 256\mu\text{m}$ 。

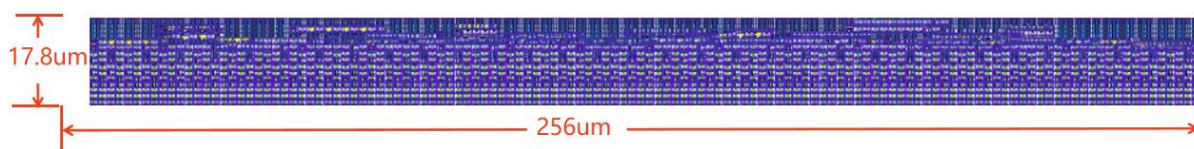


图 3-26 全 16T 加法器树版图

通过对加法器树的后端仿真, 我们可以获得其延时和功耗的数据, 如图 3-27 所示。这些数据显示, 方案二的延时基本在 1.2ns 以上, 相比于方案一, 方案二的延时较大, 但功耗表现更为优秀。通过计算得到, 当工作电压达到 0.9V 时, 加法器树的能效比达到最高。这表明在这个电压水平下, 功耗与性能之间达到了最佳平衡, 使得每单位能量消耗获得的计算性能最大化。

与方案一相比, 方案二虽然在速度上较慢, 这主要是由于其基本单元的驱动能力弱。然而, 在流水线式的存算架构中, 只要计算能够在一个周期内完成, 延时就不是最关键的因素。因此, 尽管方案二的延时较长, 但只要它不超过所设定的周期时间, 它在存算架构中的应用是可行的。这意味着在特定的应用场景下, 方案二的较高延时可以

被容忍，特别是当面积和功耗更为重要时。

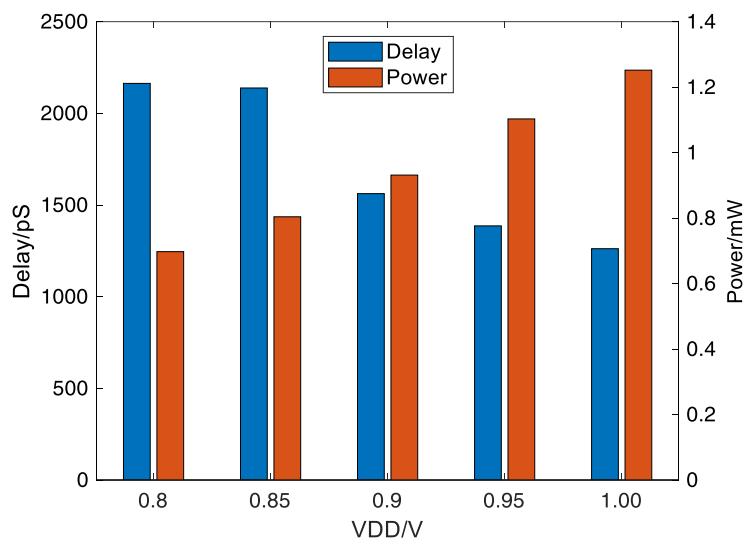


图 3-27 16T 加法器树的延时与功耗仿真图

在 0.9V 的工作电压下，图 3-28 展示了方案一与方案二的面积、功耗、延时对比。其中，面积数据是基于方案二新架构的版图计算得出的。从图中可以看出，方案二的新架构在面积上相比方案一减少了 13.2%，在功耗上降低了 20.9%，额外缓冲器的成本可以忽略不计。尽管延迟增加了 25%，但在采用流水线技术的存算架构中这样的增加是可以接受的。

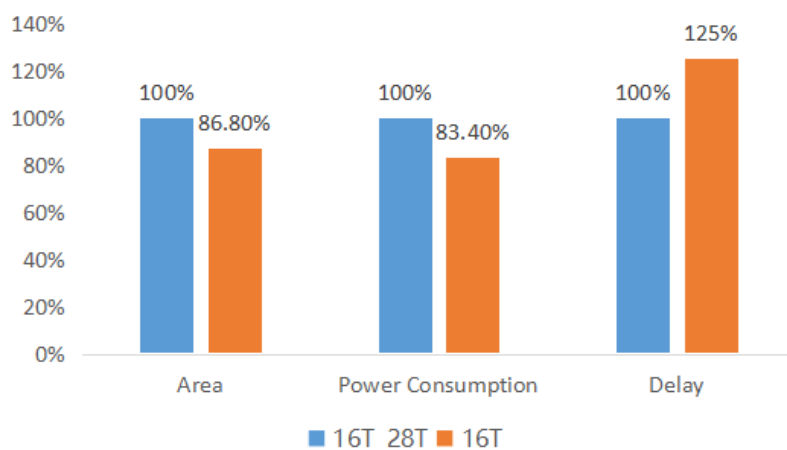


图 3-28 方案一与方案二性能指标对比

3.3.5.4 混合华莱士树结构的加法器树仿真结果

华莱士树结构是一种计算并行的进位消除加法器架构，它通过多级的并行计算结构来避免传统二进制加法器中的串行进位问题，因此在处理高位宽度输入时，其延迟时间的优势尤为明显。在设计混合华莱士树结构的加法器树时，与全 16T 加法器树结

构一样，需要依据仿真结果来优化架构设计。这涉及到确定行波进位加法器（RCA）在加法器树中使用的具体级数。

以 256 组 4 位数据相加为例，直到加法器树的第 3 级，才会出现 7 位数据相加的情况。因此，仿真从使用 3 级 RCA 开始，图 3-29 展示了使用 3 级到 5 级 RCA 的混合华莱士加法树在延时和功耗方面的仿真结果。结果显示，使用不同 RCA 级数的加法树在延时上相差不大。根据图 3-24 的仿真数据，本方案中三种结构的延时均高于方案一。在功耗方面，采用四级 RCA 的加法树功耗最高，而五级 RCA 的功耗最低。然而，基于先前的仿真数据，即便是功耗最低的五级 RCA 加法器树，其功耗也仍然高于方案一。因此，使用不同 RCA 级数的加法树在延时和功耗方面的性能表现均不如方案一。

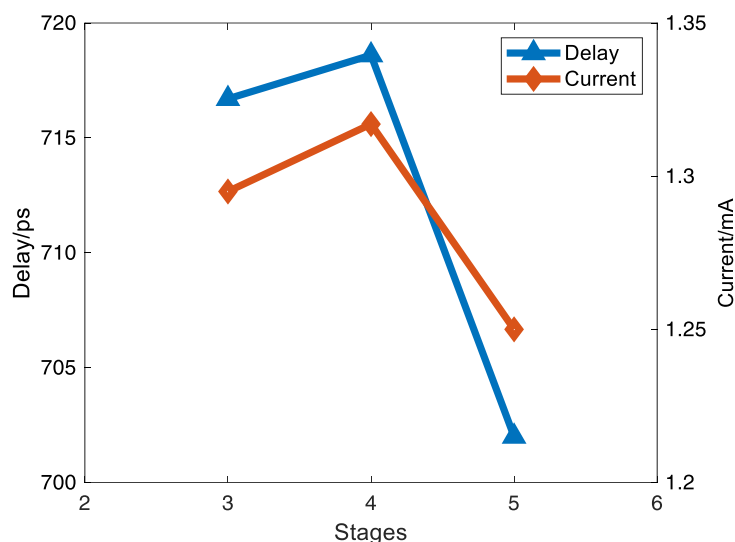


图 3-29 混合华莱士树结构的加法树延时及电流仿真

这种情况可能是由于 RCA 在此工艺和电压下所增加的进位延时，相对于华莱士树结构增加的级数所带来的延时降低并不显著。传统的二叉树加法器树结构仅需 8 级就能得出计算结果，而混合华莱士树结构可能需要 11 到 12 级的加法器才能输出最终结果。

我们可以得出结论，尽管混合华莱士树结构在某些情况下能够优化加法器树的性能，特别是在并行处理高位宽度数据时，但在当前的仿真条件下，即 28nm 工艺和 0.9V 工作电压下，它并不适合用于存算架构，其功耗和速度优势无法充分发挥，从而无法满足存算架构对于高能效的要求。

因此，为了更全面地评估混合华莱士树结构加法器树的性能，本文在 65nm 工艺和 1.2V 的工作电压条件下，使用多组随机数据对两种加法器树——混合华莱士树结构和

传统的二叉树结构——进行了仿真对比研究。如图 3-30 所示,仿真结果显示,在这些特定条件下,与传统的二叉树加法器树相比,混合华莱士树结构的加法器树能够显著提升运算速度,将延时减少到原来的 50%,而相对功耗的增加不到 10%。

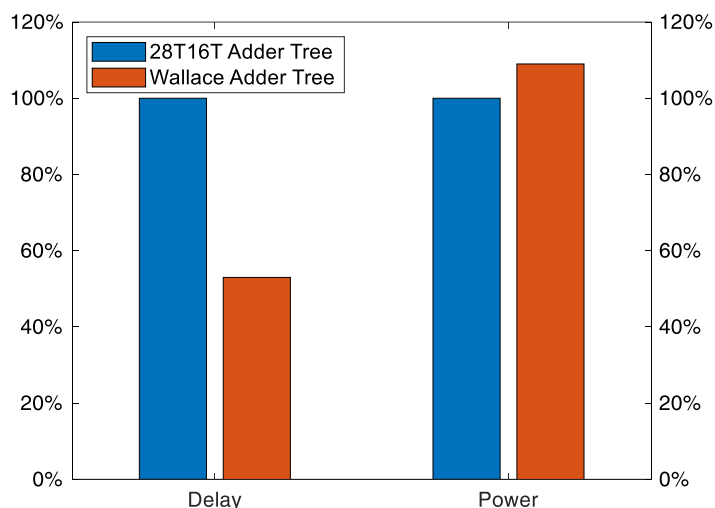


图 3-30 65nm 工艺下加法器树延时与功耗仿真

这些结果说明了在设计高性能加法器树时考虑工艺和电压条件的重要性。不同的工艺和电压条件下,加法器树的性能可能会有显著差异。因此,在选择特定的加法器树结构时,应根据目标应用的性能要求和工艺条件来做出明智的决策。在某些条件下,混合华莱士树结构可能是最佳选择,尤其是在那些对延时特别敏感而对功耗增加容忍度较高的应用场景中。

3.3.5.5 加法器树仿真总结

方案一是 28T 与 16T 交替的加法器树,表现出延时和功耗之间的最佳平衡。这种方案可能适合于那些需要在快速运算和低能耗之间寻找平衡点的应用。由于其较好的综合性能,方案一在多种不同的存算架构设计中都可能是一个可行的选择。

方案二为改进后的 16T 加法器树,它的特点是具有最低的功耗,这使得它特别适用于那些对延时不太敏感但对能耗有较高要求的场景。例如,在某些可穿戴设备或长期运行的低功耗系统中,方案二可能是一个理想的选择,因为在这些应用中,节约能源和延长电池寿命往往比计算速度更为重要。

方案三为混合华莱士树结构,在本文设计的存算架构中不适用,但它适合于大尺寸制造工艺条件下对延时敏感的应用场景。在需要较快运算速度的高性能计算环境中,如某些高频交易系统或实时数据处理应用,方案三可能更加合适。虽然其功耗可能较

高，但在这些场景下，降低延迟的重要性远超过节省能耗的考虑。

这些结论表明了在设计加法器时必须考虑特定应用场景的需求。不同的设计方案可能会因为其独特的性能特点而适用于不同的应用环境。因此，在选择最适合的加法器树结构时，必须根据所面临的具体需求和限制来做出决策。

3.4 小结

本节首先介绍了面向 CNN 的存算一体架构模型，强调了存算单元和加法树在存算架构中的重要性。接着，对本文提出的创新点进行了介绍，提出了一种 4T 可配置运算逻辑的存算单元，详细介绍了这种存算单元的工作原理和版图设计，并进行了后端仿真分析。随后，根据高能效的要求，提出了三种不同的加法器树方案，最后根据仿真结果确定了加法树的最优结构，并比较了各方案的优缺点。

第4章 基于 SRAM 的可配置逻辑的存算一体架构及电路设计

4.1 可配置逻辑的存算一体技术基本架构

本文提出了一个基于 6T SRAM 的逻辑运算可配置的高能效的 CIM 架构，支持乘法操作和布尔逻辑的计算。如图 4-1 所示，本设计主要分为两个部分：一部分是外围电路部分，由译码器模块、输入控制模块、写入/读出电路模块以及时序控制电路模块构成；另一部分为存算一体核心模块，也就是存算内计算宏。存内计算宏被分为 64 个 bank，每个 bank 中包括一个存储计算（SRAM-CIM）阵列模块、一个加法器树模块和一个对部分和进行移位累加的累加和模块，这三个模块被分为四级流水线，存储计算阵列模块和加法器树为一级流水线，累加和模块内有三级流水线，计算操作的时钟频率为 500MHz。

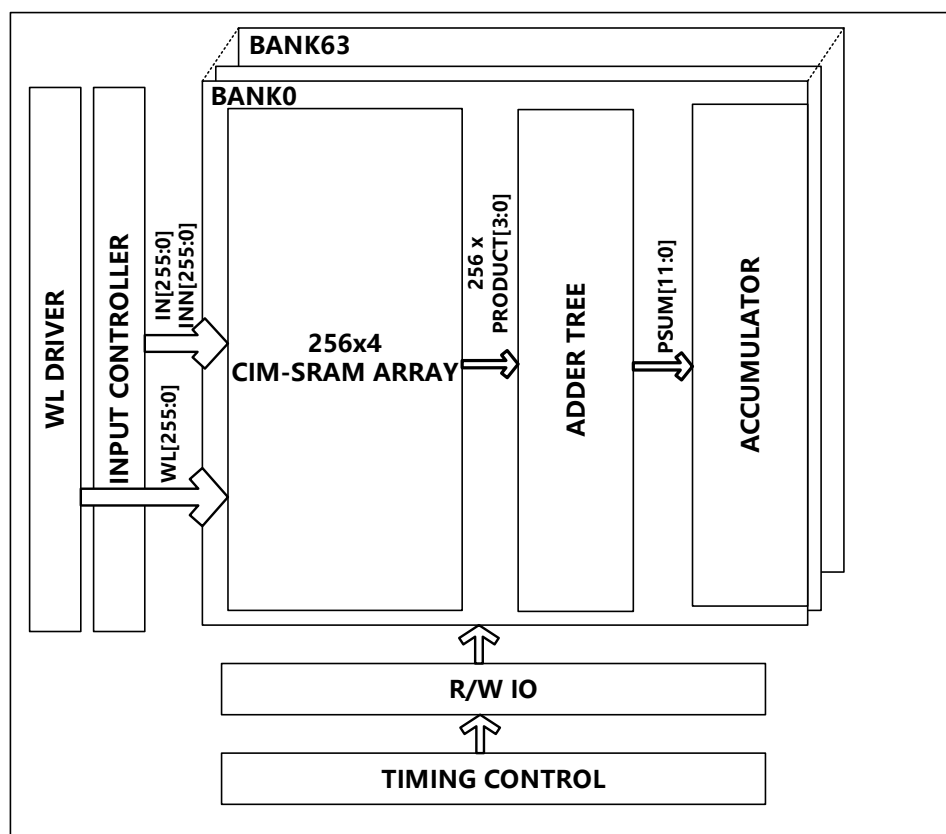


图 4-1 存算一体架构设计图

本设计可支持两种工作模式，一种是存储模式，一种是计算模式。存储模式下的 SRAM 阵列的功能与普通的 SRAM 阵列一致，主要完成数据的读写以及存储，权重的写入及更新便是在这种模式下进行。计算模式主要是用于完成数据的多种计算，最多

可支持 256 个激活因子输入、64 个计算结果输出。本设计能够完成 4bit/8bit 的有符号/无符号数输入与 4bit 的有符号/无符号数权重的乘加计算，除此之外，还能进行布尔运算，完成位的 XOR、OR、AND 逻辑运算。

在计算模式下，如果是 4bit 的输入与 4bit 的权重相乘加，首先输入数据的最高位 (MSB)，与存储在 SRAM 中的权重与输入的数据完成运算，运算完成后得到 256 组 4bit 的中间积，接着将部分积送入加法器树，输出部分和，最后部分和进入累加器完成移位相加操作，下一个周期输入次高位的输入因子继续以上操作，4bit 的输入则需要经过 5 个周期得到最后的结果。

4.2 SRAM-CIM 阵列设计

每一个 bank 中，存算阵列由 256×4 个 SRAM-CIM 构成，每个 SRAM-CIM 基本单元由 6T SRAM 和可配置的运算逻辑单元 (Reconfigurable Logic Operation Unit, RLOU) 组成，如图 4-2 所示，每个 SRAM 单元的 Q、QB 端与 RLOU 相连，以完成存内运算的功能。

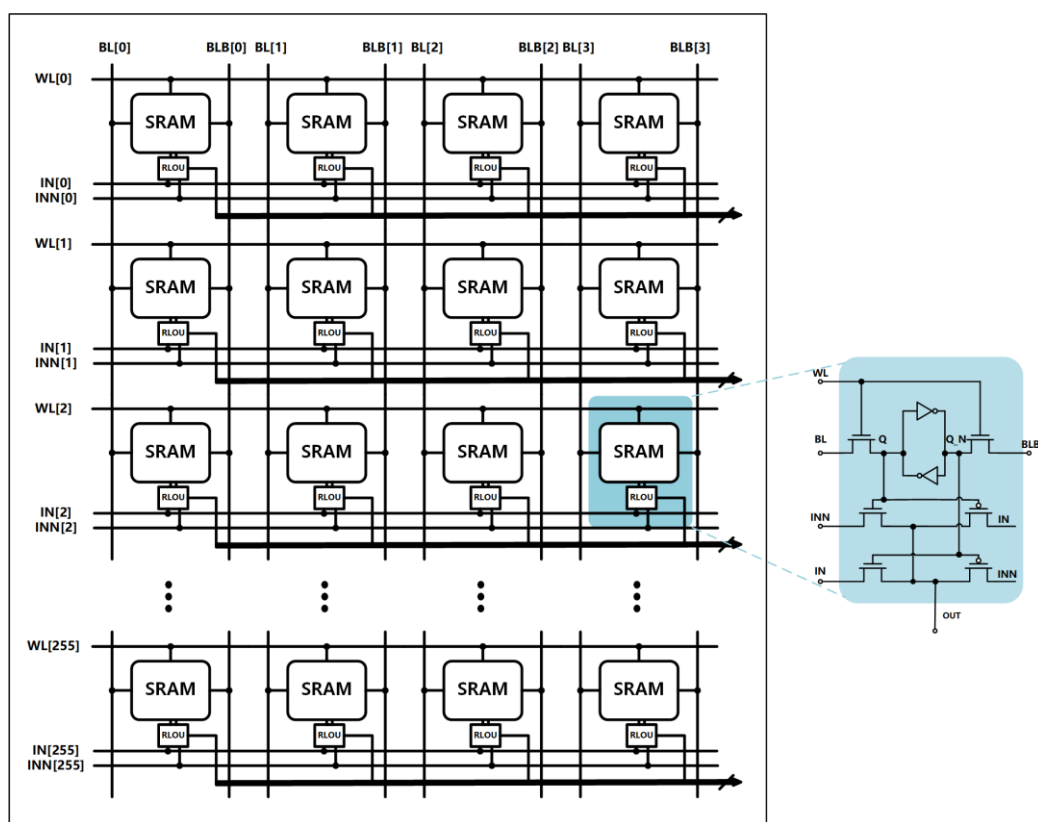


图 4-2 SRAM-CIM 阵列架构

SRAM-CIM 阵列可以工作在两种模式下，一种是正常的 SRAM 存储模式，一种是

使用到 RLOU 模块的计算模式。存储模式时，主要工作的是 SRAM 单元，IN、INN 默认为低电平，此时计算单元不工作，只有静态功耗，权重信息可以通过存储模式写入阵列中，SRAM 阵列的读写速度可以达到 1GHz。计算模式时，256 组输入激活因子通过并行输入的方式输入计算单元，此时 IN、INN 输入有效，存储在 SRAM 单元里的权重信息通过 Q、QN 输入到 RLOU 模块中，RLOU 模块完成计算后输出结果到加法器树单元，在一个周期内存算阵列可以完成 $256 \times 4 \times 64$ 次运算。

4.2.1 输入控制模块

RLOU 电路要完成可配置的逻辑运算，便要使用输入控制模块对输入数值进行处理。输入控制模块的控制逻辑如表 4-1 所示，两比特的控制信号完成对输入数据的处理，INPUT 为输入数据，INPUT_N 为输入数据的反相，IN、INN 为输出端，与 RLOU 单元相连。当控制信号为 11 时，运算逻辑为异或，此时 IN、INN 端口保持不变，这种运算可用于两张图片汉明距离的计算；当控制信号为 10 时，运算逻辑为或，此时 IN 端口保持不变，INN 端口始终输出高电平；当控制信号为 01 时，运算逻辑为与，也就是相乘，此时 INN 保持不变，IN 端口输出低电平；当控制信号为 00 时，不做运算，IN 端口保持低电平，INN 端口保持高电平。

表 4-1 输入控制模块的控制逻辑表

控制信号	运算逻辑	IN	INN
11	XOR	INPUT	INPUT_N
10	OR	INPUT	1
01	AND	0	INPUT_N
00	RST	0	0

根据表 4-1 所示逻辑，可以推断出 IN 与 INN 关于控制信号 CONTROL[1:0]和输入 INPUT 的逻辑表达式：

$$IN = CONTROL[1] \cdot INPUT \quad (4-1)$$

$$INN = \overline{CONTROL[0] \cdot INPUT} \oplus CONTROL[1] \quad (4-2)$$

通过逻辑表达式可以画出输入控制模块基本单元的电路图，如图 4-3 所示。输入控制模块主要由标准单元库搭建，因此可以将输入控制模块与输入驱动模块结合在一起，以提高模块的面积效益和简化整体存算架构的复杂性。在控制单元中，INPUT 是输入的激活因子，两位控制信号负责处理输入数据，IN、INN 是输入控制模块的输出，它们连接到 CIM 阵列。通过结合输入控制模块和 RLOU 模块的功能，就可以配置存算

架构执行多种操作，包括 XOR、OR、AND 和重置 (RST)。这些操作的多样性使得存算架构具有高度的灵活性和适应性，能够满足各种计算需求。特别是在处理复杂的逻辑运算和数据处理任务时，这种多功能性显得尤为重要。

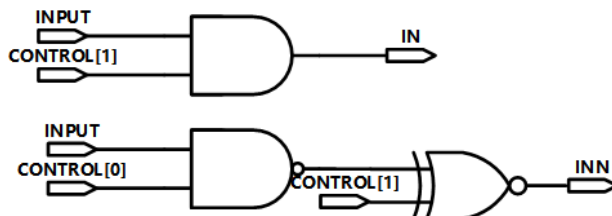


图 4-3 输入控制模块基本单元

4.2.2 译码器电路设计

本设计中，SRAM 的数据位宽为 64 位（64 个 bank），每个 bank 包括 4bit 数据，阵列大小为 64Kb，因此需要 10bit 的地址信号进行寻址。

10bit 地址信号中低两位用于进行位寻址，也就是在一行中寻址 4bit 数据中的一个，高 8 位用于行寻址。低两位可直接解码出列地址，行地址由于位数较多，使用二级译码器进行译码。这里将 8 位地址信号分为 2-3-3 的组合，分别解码出 4 位、8 位、8 位的独热码，再完成 256 位行选择信号的转换。译码器电路图如图 4-4 所示。

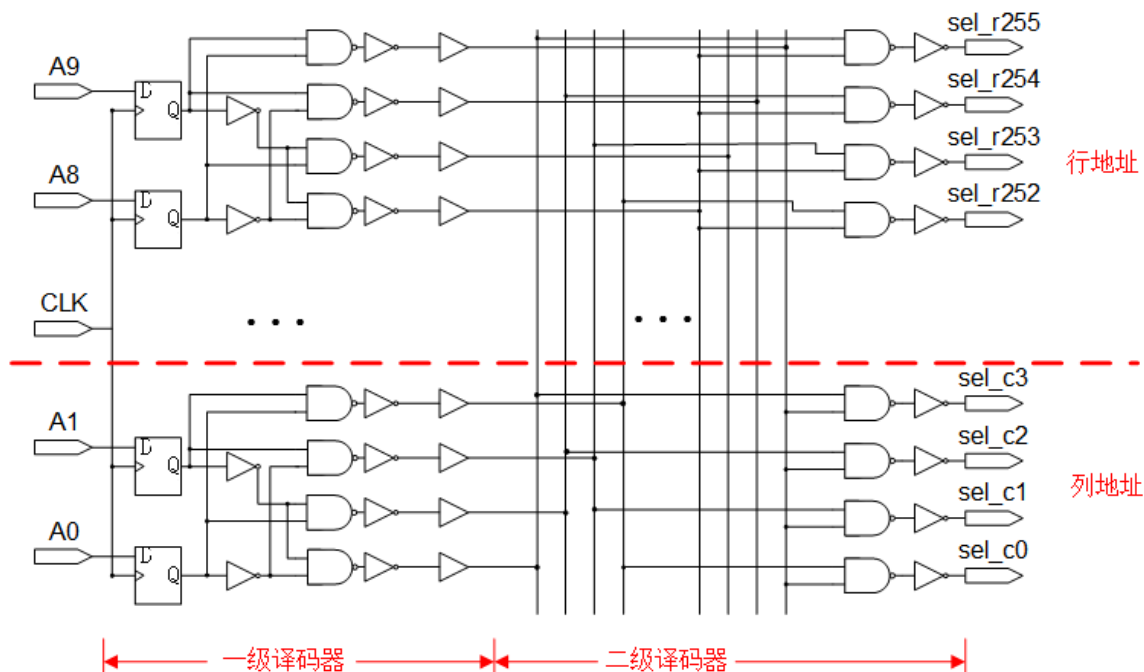


图 4-4 译码器电路图

4.2.3 灵敏放大器设计

SRAM 阵列的读出数据是根据 BL、BLB 上的电压差得到的，因此读出电路一般使

用灵敏放大器（Sense Amplifier, SA）对位线上的电压进行放大，得到读出结果。本文采用的是经典的5管运放，如图4-5所示。其中，BL、BLB与M1、M2相连，M3、M4是镜像电流源，SAE（Sample Enable）是采样信号，用于控制尾管M5的开关。

当读操作时，BL、BLB上出现电压差，SAE开启，灵敏放大器便会放大两条位线上的差分信号，输出读出结果。

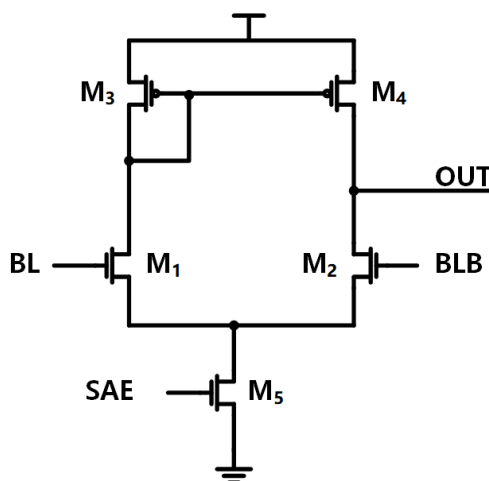


图 4-5 灵敏放大器电路图

4.2.4 读写自适应模块设计

在SRAM中，时序控制电路的设计至关重要。为了降低功耗，目前的读写控制电路一般会使用自适应的方式追踪读写的延时，以及时控制WL和预充电（precharge）信号的开关。追踪电路一般由一系列冗余的SRAM阵列构成，模拟SRAM阵列中的读写过程，冗余SRAM单元里的值始终保持低电平。

读写自适应模块的结构如图4-6所示，TRKBL为BL追踪信号，SRAM_enb为SRAM的使能信号，低电平有效，Write_en为读使能信号，高电平有效。TRKBL与冗余阵列中部分SRAM单元的BL线相连，这是为了模拟BL线上电容的大小，连接SRAM的具体数目由仿真决定。当SRAM_enb为高电平时，表示SRAM未被选中，SAE为低电平，读操作无效。当SRAM_enb为低电平，Write_en为高电平时，读操作有效，SAE开始采样，此时由于WL开启，TRKBL通过dummy SRAM连接到VSS，当放电结束TRKBL降为低电平时，Precharge_n信号有效，SAE关闭，WL降为低电平，读操作结束。

这种设计方式既可以保证SRAM的读写操作顺利完成，还可以及时关闭字线和读出电路，节省芯片的功耗。

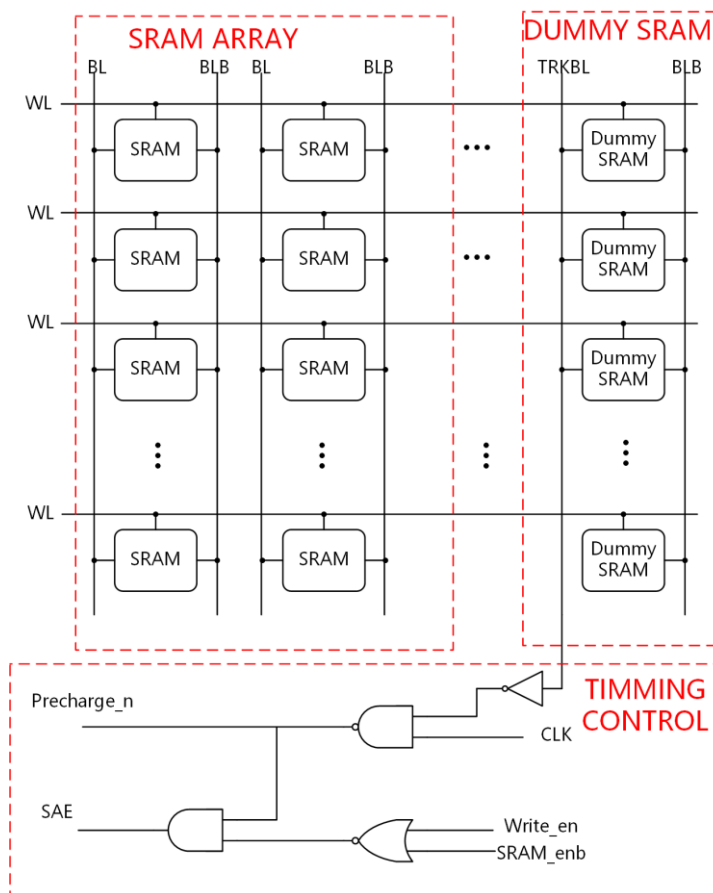


图 4-6 读写自适应模块结构示意图

4.3 加法器树结构设计

在设计面向乘累加运算的SRAM存内计算宏电路时，其中并行加法电路的设计显得尤为重要。这是因为，并行加法器不仅在整个宏电路中占据了相当的面积和功耗，而且其延时特性直接影响到存内计算宏电路的工作频率，进而决定整个系统的运算效率。

考虑到设计采用的时钟频率为500MHz，这意味着每个时钟周期为2ns。在这样的时钟约束下，需要选择一种能够在2ns内完成运算的加法器树。根据第三章的内容，在28nm工艺下，能够满足这一性能要求的有两种加法器树：28T与16T混合加法器树和改进后的16T加法器树。这两种加法器树中，改进后的16T加法器树有更低的功耗和更小的面积，本存算架构的目的是提高能效比，因此在这里选用的加法器树是第三章中提出的基于全16T全加器的加法器树结构。

4.4 移位累加模块设计

累加器模块是高性能计算系统中的一个关键组成部分，其主要功能是对部分和进

行移位相加，其结构框图如图 4-7 所示。在累加器模块中有三级流水线，以优化数据处理速度和提高整体系统效率。第一级是输入寄存器，主要工作是对输入数据进行采样储存，并完成符号扩展与数据相加；第二级是移位寄存器，由于输入数据是从高位到低位依次输入的，因此在每一次部分和相加完成后，都需要将结果向左移动一位，以便正确地与下一位数据进行相加；第三级是输出寄存器，其功能是对结果进行储存。累加器模块最高可支持 20bit 数据的输出。

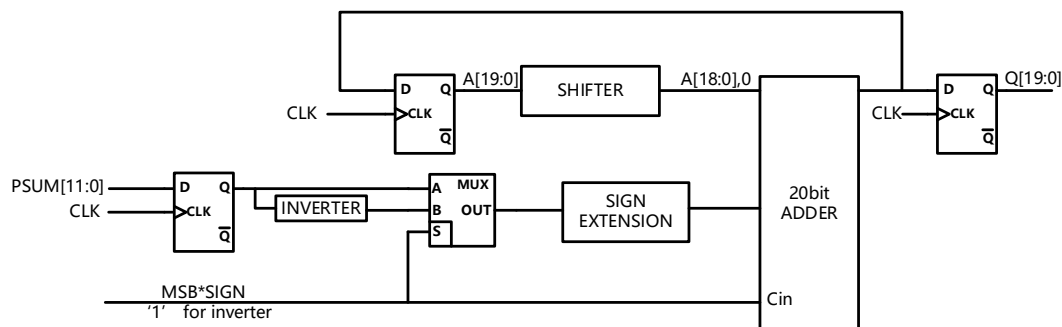


图 4-7 累加器结构框图

如果是乘累加运算，当输入激活因子为有符号数时，需要对最高位的部分和进行特殊处理，即执行取补码的操作，也就是将数据取反后再加一，以确保能得到正确的有符号数结果。累加器部分电路由 Verilog 代码综合而成，通过这种编程方式实现的累加器模块不仅可以灵活控制操作数的位宽，而且确保了处理速度和精度，满足了本设计中存算一体架构的计算需求。

4.5 可配置的逻辑运算应用方法

本论文设计的存内计算可以进行不同的逻辑运算，本节主要介绍可配置逻辑运算存算一体架构的应用方法，具体介绍面向不同场景时对数据的配置。

4.5.1 面向 CNN 的存内计算

卷积神经网络是典型的人工智能算法，卷积运算就是将大量输入与权重相乘后相加，形成网络的输出。本论文所提出的存内计算宏电路主要针对人工智能算法中的乘累加运算进行了专门的设计和优化。在这类算法中，权重和输入数据的位宽对数值的表示范围有着直接影响，进而决定了算法的表达能力和推理精度。通常情况下，更大的数据位宽可以提供更高的算法推理精度，因为它允许更精细的数值表示。然而，增加数据位宽也会增加存储和计算的需求，对存内计算宏电路的设计提出了更高的要求。

大多数现有基于 SRAM 的存内计算宏电路通常仅支持最大 4 比特的权重和输入数

据位宽，这在某些应用中可能限制了算法的推理精度。为了克服这一限制，本论文提出的存内计算宏电路支持更广泛的输入数据位宽范围，从1比特到16比特不等，同时支持4比特或8比特的权重数据位宽。这种可扩展性使得存内计算宏电路能够更好地适应各种不同的应用场景，特别是那些对推理精度有较高要求的场合。

输入数据通过位串行的方式输入到SRAM阵列中。这意味着，如果输入数据的位宽为4比特，则需要4个时钟周期来完成整个输入过程。具体来说，输入数据从高位至低位依次输入，每个时钟周期处理一个比特。

权重数据则通过写入SRAM的方式预先加载。当权重数据的位宽为4比特时，每个SRAM阵列的Bank中的一行存储一个权重数据。而当权重位宽为8比特时，相邻两个Bank的同一行共同存储一个权重数据，其中一个Bank存储高4比特，另一个Bank存储低4比特。

在计算模式下，首先把运算逻辑切换为AND，并输入256个输入激活因子。输入数据与存储在SRAM中的权重信息完成相乘运算，输出256组4比特的部分积，部分积通过改进后的16T的加法器树进行累加，可得到一个12比特的部分和。由于这是第一个时钟周期得到的结果，所以需要对部分和进行符号扩展和数据储存，累加器便是实现此功能的模块，运算时序图如图4-8所示。用PSUM₀、PSUM₁、PSUM₂、PSUM₃表示四个周期的结果，整个计算过程可以用下面的公式表示：

$$MAC_{result} = 2^3 \times PSUM_0 + 2^2 \times PSUM_1 + 2^1 \times PSUM_2 + 2^0 \times PSUM_3 \quad (4-3)$$

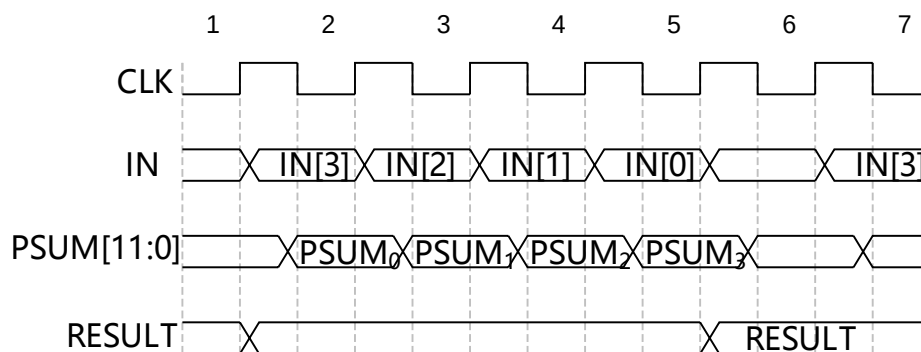


图 4-8 累加器运算时序图

4.5.2 面向汉明距离的存内计算

汉明距离是一种衡量两个相同长度的序列之间差异的度量方法，在信号处理和模式识别领域中被广泛应用^[46, 47]。汉明距离的定义是：给定两个等长的数组或字符串，其汉明距离是对应位置上不同码元的数量。换句话说，它衡量了需要改变多少个码元

(比如二进制位) 才能将一个数组变换为另一个数组。

对于两个二进制字符串 $A[i]$ 和 $B[i]$, 其汉明距离 $d_H(A, B)$ 可以表示为:

$$d_H(A, B) = \sum_{i=1}^n (A[i] \oplus B[i]) \quad (4-4)$$

这里 $A[i]$ 和 $B[i]$ 是数组 A 和 B 在第 i 位置上的码元, n 是数组的长度, 方括号内的表达式如果为真 (即 $A[i]$ 和 $B[i]$ 在该位置上不同) 则结果为 1, 否则为 0。然后对所有这些值求和, 得到的总和就是两个字符串之间的汉明距离。从上述公式可以看出, 汉明距离的计算就是基于异或逻辑运算。

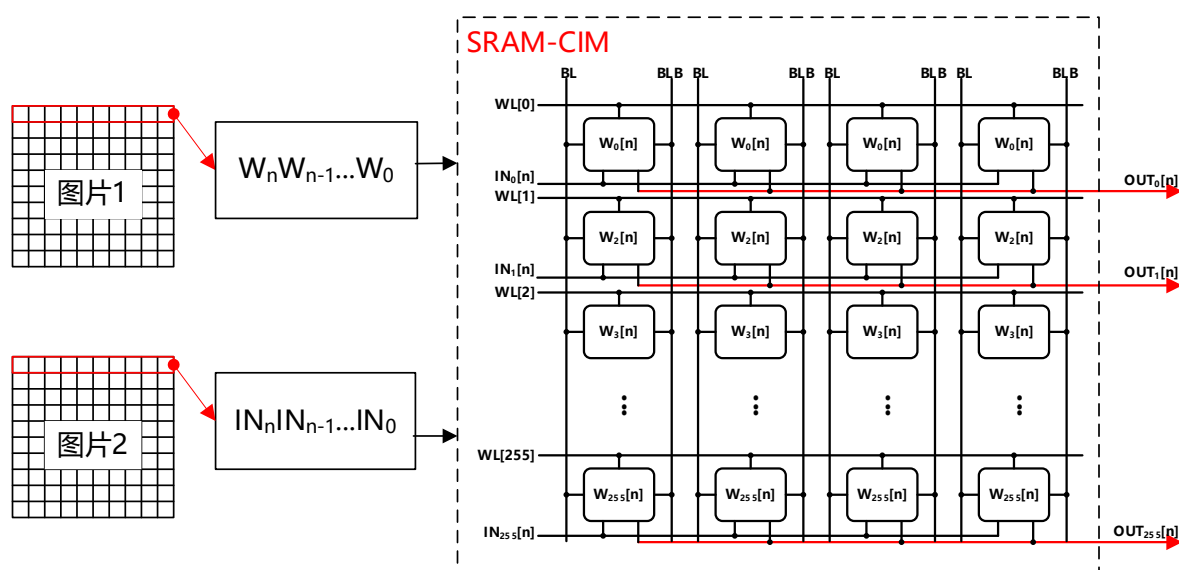


图 4-9 汉明距离运算结构图

本文设计的存算架构可以被配置为异或逻辑运算, 通过结合加法器树的功能, 能计算出两张图片的汉明距离。图 4-9 展示了存内计算汉明距离运算过程。首先, 将被检测图片加载进存储阵列中, 在储存图片数据时, SRAM 阵列中每个 Bank 的同一行存储同一比特图片数据, 这意味着, 每个 Bank 里一行 4 比特的数据是一样的, 因此需要将同一组数据输入四次。根据表 4-1, 当存算架构被配置为异或运算时, INN 为 IN 的反相, 在 SRAM-CIM 的一个 bank 中, 输入输出真值表如表 4-2 所示, 如果把异或运算的结果视为有符号数, 理论上的结果将是表内结果的负数。因此为了得到正确的汉明距离, 只需要将输出结果进行取补码操作。

表 4-2 异或逻辑输出输出真值表

IN	INN	Weight[3:0]	OUT[3:0]
0	1	0000	0000
0	1	1111	1111
1	0	0000	1111

1	0	0000	0000
---	---	------	------

接下来的步骤是将对比图片作为输入数据，在这个阶段，权重的符号控制信号被设置为1，这意味着系统将其处理为有符号数的相加操作，在累加器模块中，将输入的符号控制信号和最高位控制信号设为1，移位信号设置为0，这表示在这个过程中执行取补码操作，但是不执行移位操作。最后，如果是4比特的输入和4比特的权重求汉明距离，经过5个周期后的结果即为两张图片之间的汉明距离。

总之，本文设计的存算架构通过巧妙地利用可配置的逻辑运算单元和加法器树的组合，能够有效地切换不同的计算，这种方法不仅快速而且精确，对于需要处理大量图像数据的应用场景尤为适用。这种设计方法不仅提高了运算的效率，还增加了架构的灵活性。通过简单的配置更改，可以使存算架构适应不同的计算需求，例如卷积操作和汉明距离的计算。这种能力在处理图像数据、进行模式识别或执行其他需要快速而精确比较的任务时尤其有价值。

因此，本文所提出的SRAM存算一体结构可以提高数据的处理效率和实时性能，可以应用在智能传感器网络、嵌入式图像处理、移动终端设备等嵌入式场景中

4.6 可配置逻辑运算的存算架构仿真分析

本节主要对上文提出的基于SRAM的可配置逻辑运算的存算架构进行仿真分析。使用的仿真软件为Cadence，输入数据均为随机数据，由Verilog代码中的\$Random函数实现，通过代码生成激励模块，激励模块与测试模块相连并进行ams仿真，时钟周期为500MHz。仿真条件为28nm工艺，0.9V工作电压和27℃的温度。

4.6.1 SRAM阵列的读写速度及功耗

SRAM的读写操作一般是一行多列进行操作，在这里我们搭建了一个64Kb的SRAM计算阵列仿真电路，这个阵列里不仅包含SRAM单元和RLOU计算单元，还集成了外围电路，如译码器、预充电电路、时序控制电路、读出电路等。这里使用500MHz的时钟对SRAM进行操作，输入数据为256组4bit的随机数据，仿真结果如图4-10所示。

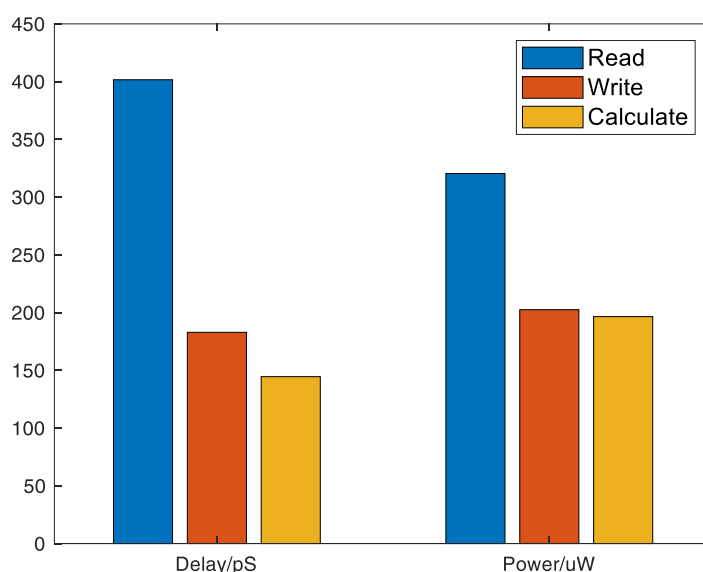


图 4-10 SRAM 阵列延时及功耗仿真

由仿真图可以观察到读写操作的延时和功耗都比较大，这是因为读写操作的时间是由位线电容的大小决定的，SRAM 阵列越大，读写时间越长。特别是读操作，由于需要额外的读出电路，其延时和功耗更加显著，相比之下，计算操作由于计算单元电路较为简单，平均功耗和延时都较小。

4.6.2 累加器仿真

累加器主要由 RTL 代码综合而成，其仿真时序如图 4-11 所示。CLK 为时钟信号，时钟频率为 500MHz。RST_N 为复位信号，低电平有效，且可以异步复位。SHIFT 为移位信号，“1”表示需要移位，“0”表示不需要移位，比如在计算汉明距离时不需要对数据进行移位。SIGN 为符号位信号，“1”表示输入为有符号数，“0”表示无符号数。VLD 信号为计算有效信号，与输入激活因子同步输入，“1”表示计算模式。MSB 为最高位信号，“1”表示输入为最高位。PSUM 为部分和，由加法器输出，Q 为累加器的输出。累加器的四个部分和分别为-100、50、-60、80，由公式(4-3)可得理论结果为 960，与累加器的输出结果一致。

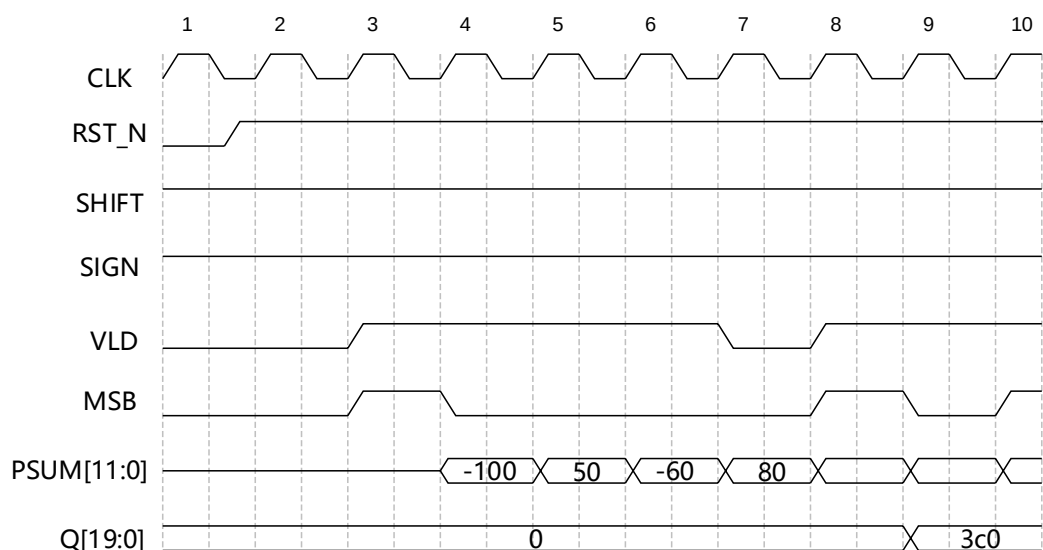


图 4-11 累加器仿真时序图

4.6.3 基于SRAM的可配置逻辑运算的存算一体架构仿真

首先绘制了如图 4-12 所示的版图，版图大小为 $890\mu\text{m} \times 553\mu\text{m}$ 。由于 SRAM 阵列较大，因此在版图绘制过程中，着重考虑了信号线的驱动问题以及寄生电容的影响。信号线有足够的驱动能力才能够有效地传输数据和控制信号到整个阵列。寄生电容可能会影响信号的传输速度和电路的整体性能。在版图设计中，我们努力最小化这些影响，以提高电路的响应速度和效率。因此，我们将时钟控制模块放置在版图的中央位置。这样做可以使信号线到达各模块的路径最短，从而减少延迟和信号衰减。鉴于降低功耗是本设计的一个主要目标，我们在版图中主要使用了高阈值电压（high VT, hvt）的 MOSFETs。这些器件在减少漏电流方面表现优异，有助于降低整个电路的静态功耗。

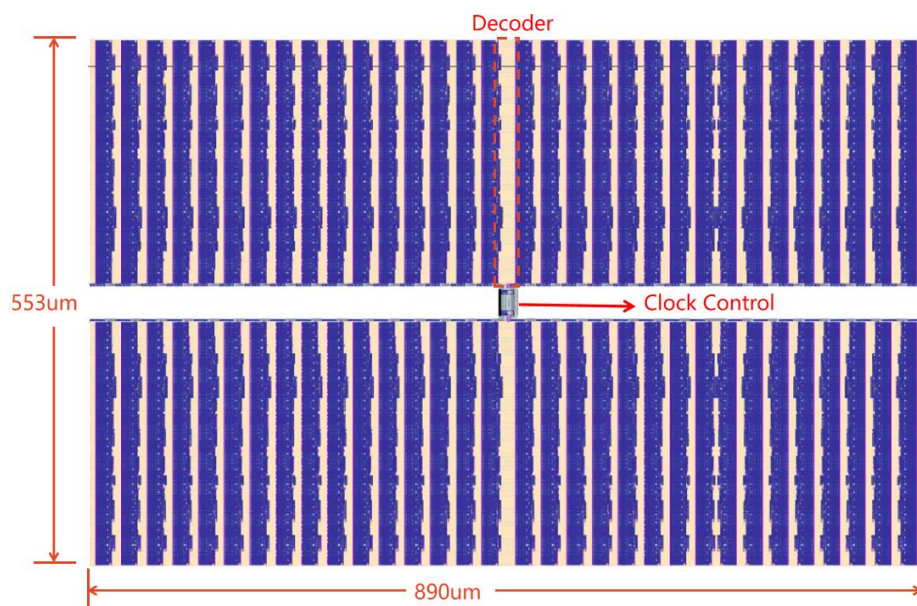


图 4-12 存算架构版图

4.6.3.1 功能仿真

在我们进行的 SRAM 仿真中，观察到读写操作可以在不到 500 皮秒(ps)内完成。基于这一发现，我们将 SRAM 的读写操作频率设定为 1GHz。但是在计算模式时，频率仍维持在 500MHz。相关的读写以及乘加操作的仿真图如下所示。

首先，进行数据写入阶段。这一阶段涉及将 64Kb 的数据分为 256×4 个部分进行写入，所有数据均为随机生成。为了验证写入的正确性，数据从 SRAM 中读出并进行核对。在验证时，需要特别关注距离时序控制电路模块最远的第一列数据，因为这列数据经历了最长的传输线路。如果第一列数据无误，即表明 SRAM 的读写操作正常。如图 4-13 所示，仿真结果显示，在不同位置写入后读出的数据与原始数据一致，验证了 SRAM 的读写功能。

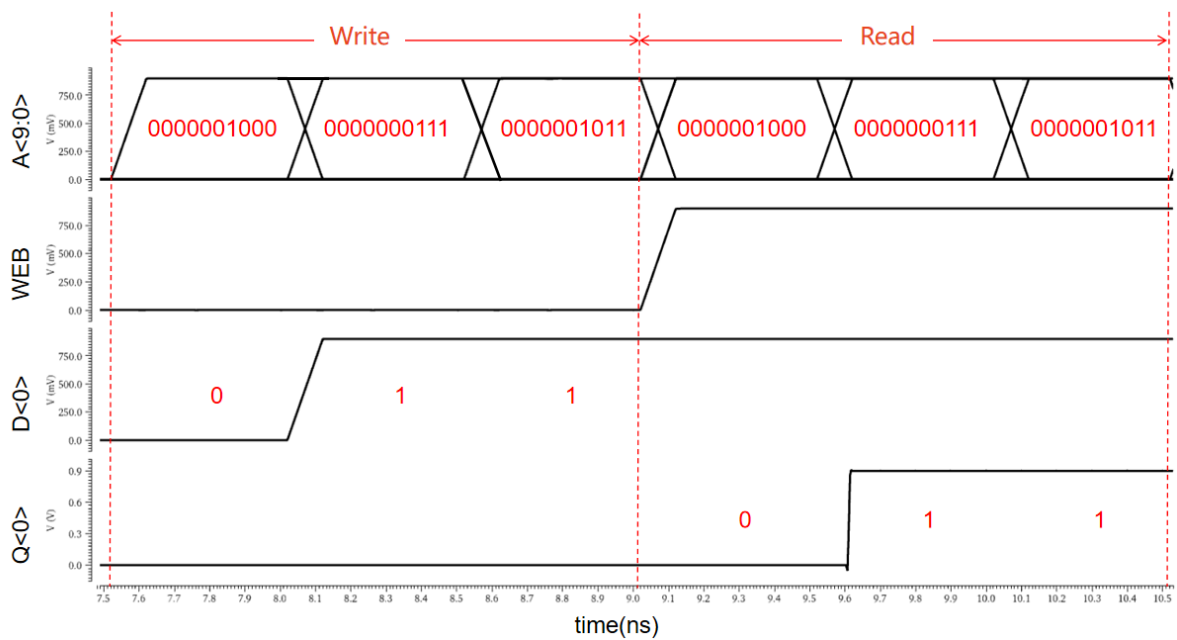


图 4-13 读写时序仿真图

在接下来的步骤中，我们对乘累加（MAC）操作进行了验证。如图 4-14 所示，首先，在第一个周期输入 256 组随机数，产生 256 组 4 比特的中间乘积。这些中间积经过加法器树处理后，得到一个 12 位的部分和，可以看出该部分和可以在一个周期内得到。四个周期后，我们获得了 4 个部分和。验证结果表明，这些部分和均正确无误。然后，这 4 个部分和被送入累加器进行处理。由于输入和权重为有符号数，最高位的部分积需进行取补码操作。一共经过五个周期，可以得到最终的计算结果。

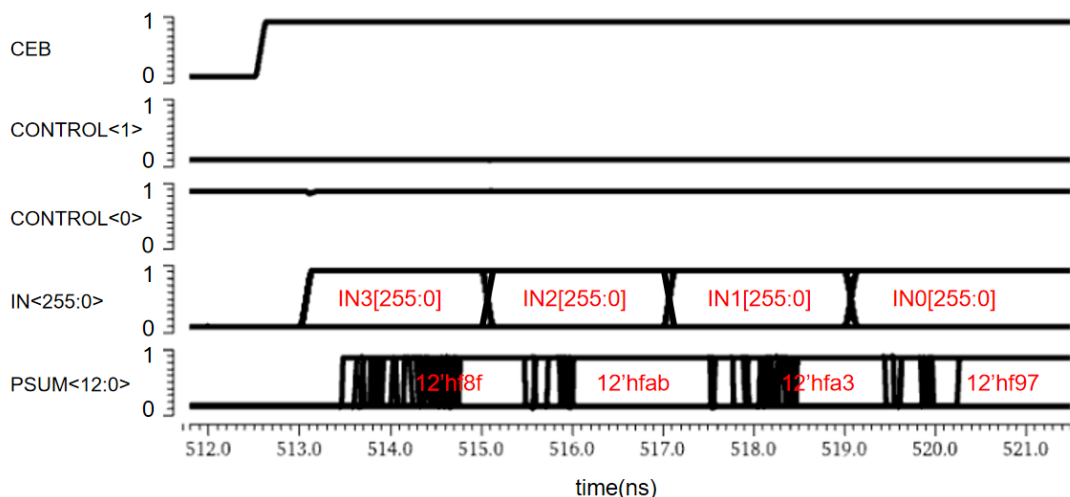


图 4-14 存算架构乘加仿真时序图

除了 MAC 操作的验证之外，我们还对异或操作进行了测试，时序图如图 4-15 所示。首先，我们改变控制信号，从而将 SRAM 的操作模式转换为执行异或逻辑操作。

在模式切换之后，我们向 SRAM 阵列输入权重数据。输入数据随后被送入 SRAM 阵列，并与权重数据进行异或操作。异或操作完成后，数据通过加法器树处理，生成了 4 个部分和。验证结果表明，这些结果均正确。

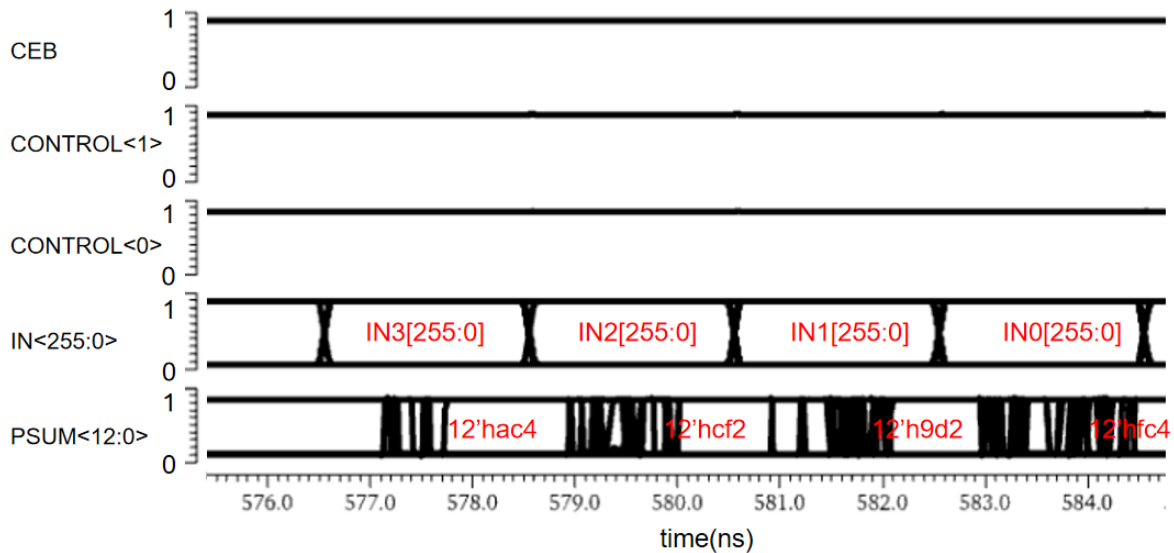


图 4-15 XOR 操作时序仿真图

4.6.3.2 性能仿真

接下来对存算架构进行性能测试。首先对存算宏中各个模块的功耗进行仿真，如图 4-16 所示。从图中可以看出，加法树的功耗占比最高，这表明加法树在整个存算架构中对功耗的影响极为显著。因此，优化加法树的功耗是很重要的。输入控制模块的功耗占比排在第二位。这主要是因为输入控制模块包含大量缓冲器，这些缓冲器在大型阵列中用于驱动信号线。

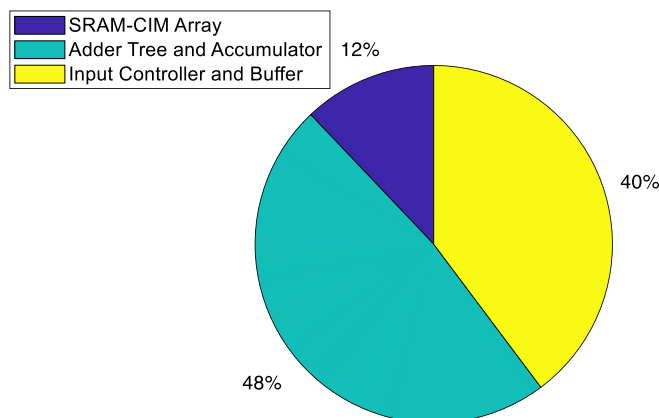


图 4-16 存算架构宏内各模块功耗占比

本文提出了一种改进后的 16T 加法树结构，旨在减少存算架构宏的面积与功耗。为了全面评估加法树结构对性能的影响，本文还构建了一个使用 28T 与 16T 交替加法树的存算架构，并采用了 28nm 工艺进行仿真。如图 4-17 所示，仿真结果表明，改进后的 16T 加法树结构减少了存算架构的功耗和面积。与采用 28T 与 16T 交替加法树的存算架构相比，本文提出的架构在功耗上降低了 12.4%，在面积上减小了 15.6%，能效比（即算力与功耗的比值）提高了 18.5%。这表明本文提出的架构能够有效提升存算的能效比和面积利用率，从而优化整体性能。

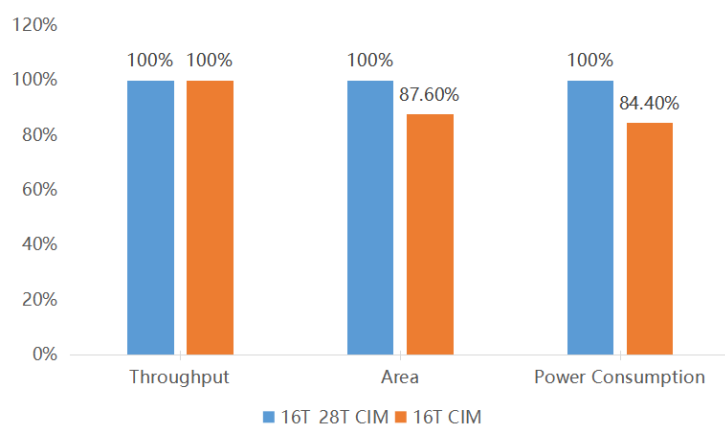


图 4-17 两种存算架构的性能对比

能效比是存算电路最重要的指标之一，能效比本质上衡量的是电路每消耗单位能量所能执行的计算量，是评估存算架构性能的重要参数，特别是在边缘设备的应用场景中。为了测试存算架构的能效比，我们进行了后端仿真，来分析不同电压条件下的算力与能效比的变化。后仿结果如图 4-18 所示。

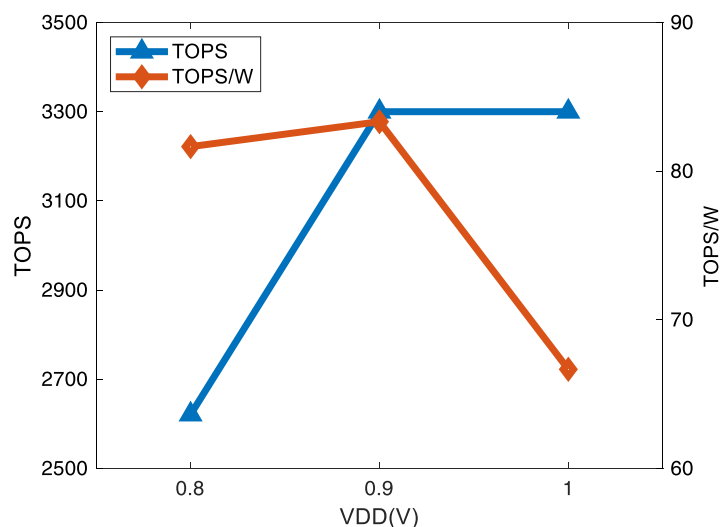


图 4-18 存算架构的算力与能效比结果

从图中展示的结果来看,存算架构的算力随着电压的增加而提升。这是因为较高的电压通常会增强晶体管的开关速度,从而提升整个电路的处理能力和速度。然而,在0.9V、500MHz的工作条件下,能效比表现得较好。这是因为,电压的升高也伴随着功耗的增加,更高的电压会导致晶体管漏电流增大以及动态功耗上升。而在低电压情况下,功耗虽然降低了,但是架构的运算速度也会变慢,降低了存算架构的能效比。

4.6.3.3 性能对比

表 4-3 展示了目前现有几个研究工作和本论文提出架构的不同指标对比,包括SRAM的类型与大小、工艺节点、工作电压、阵列大小、存算架构宏面积、计算方式、支持的运算类型、输入及权重精度、时钟周期、算力以及能效比。其中文献[23]由28nm工艺复现,测得性能指标。

表 4-3 存内计算宏性能指标对比

	[17]	[48]	[19]	[23]	<i>This work</i>
Bitcell	6T	6T	6T	6T	6T
Technology(nm)	28	65	28	28	28
CIM Macro Type	Analog	Digital	Digital	Digital	Digital
Supply Voltage(V)	0.7-0.9	0.6-0.8	0.8	0.9&0.8	0.9
Array Size	384Kb	16Kb	32Kb	64Kb	64Kb
Macro Area(mm ²)	1.4	0.2272	0.03	0.6	0.5
Compute Circuits	Ph-ADC	-	Dynamic Logic Compute Circuit	CMOS Static Logic	CMOS Static Logic
Support Bitwise Operation	AND	AND	AND, XOR, OR	AND	AND, XOR, OR
Input Bits	4b/8b	1b-16b	1-8b	1-8b	1-8b
Weight Bits	4b/8b	4b/8b/12b/16b	1b/4b/8b	4b/8b/12b/16b	4b/8b/12b/16b
Cycle Time	4ns(4b/4b) 7.2ns(8b/8b)	-	3ns/Input bit	10ns(4b/4b) 18ns(8b/8b)	10ns(4b/4b) 18ns(8b/8b)
Throughput(GOPS)	-	567(1b/1b)	-	3300(4b/4b)	3300(4b/4b)
Energy Efficiency(TOPS/W)	77.3(4b/4b)	117.3(1b/1b)	27.38(8b/8b)	58(4b/4b)	83.33(4b/4b)

本文提出了一种基于SRAM的全精度可配置逻辑运算CIM架构,采用28纳米工艺。与文献[17]采用的模拟存算技术不同,本文采用的是数字存算技术。数字存算技术

的优势在于能够支持更高精度的权重和输入，并且能够灵活地扩展计算位数。相比与现有工作[19]中采用的动态逻辑计算单元，本文提出的静态结构更有利于数据的保持，且控制电路更简单，有更高的并行性和计算效率。

此外，这项工作在 28nm 的工艺下实现了 83.3 TOPS/W 的能效比，与参考文献[23]中提到的 28nm 工艺的存算架构相比，能效比提高了 43%。这得益于采用基于 16T 的加法器树设计，与[23]中使用的加法器树（用 28nm 工艺搭建的加法树）相比，面积减少了 16.6%，功耗降低了 13.2%。本文提出的 CIM 宏还可以配置逻辑运算以执行 MAC 操作以及其他布尔运算，如 AND、OR、XOR 等。这种灵活性使系统能够根据特定应用的具体需求适应不同的计算任务，优化资源利用，例如可以完成向量矩阵乘法/向量哈达玛积（VMM/VHP）或图像信号处理中的汉明距离计算等任务。

4.7 小结

本节详细介绍了一种基于 SRAM 的可配置逻辑运算的高能效存算一体架构。首先，文章对存算架构的不同模块进行了详细描述，包括每个模块的构成、优缺点，并对 SRAM 的性能指标进行了深入分析。接着，重点介绍了可配置逻辑运算架构的工作原理和工作过程，这是架构中的关键组成部分。随后，文章通过仿真展示了各个模块的功能和效率，特别关注了在 SRAM 单元内数据的稳定性，以及读写操作和计算操作所产生的功耗及延时。最后，文章通过性能仿真对整个架构进行了评估，并将其在算力、能效比和面积等方面与其他类似工作进行了对比分析。

第5章 结论与展望

5.1 论文工作总结

存算一体是一种新兴的计算架构，它的核心目标是通过在存储子系统中整合计算功能，以此来显著提升计算系统的性能和能效比。这种技术的应用范围非常广泛，涵盖了从人工智能到机器学习，再到物联网等多个领域。在这些应用中，存储子系统不仅仅负责数据存储，还需要执行各种计算操作或函数，这些操作的多样性是 CIM 技术的一个关键特点。

本篇论文提出了一种基于 SRAM 的可配置逻辑运算的高能效存内计算架构，主要工作总结如下：

1. 本文开始于对国内外关于存算架构中存储单元设计的研究现状进行调研。在此基础上，对现有的存算架构进行了详细分析，并指出了目前存算架构设计中存在的主要问题。
2. 本文的核心贡献之一是提出了一种新型的可灵活配置运算逻辑的计算单元结构。这种计算单元不仅具有面积小、功耗低的特点，而且灵活性高，可根据不同的应用需求进行配置。特别地，这种计算单元非常适合用于卷积神经网络和汉明距离的计算。这样的设计不仅提高了计算效率，还扩展了存算架构在各种应用中的适用性。
3. 文章着重考虑了加法器树设计对存算架构的面积和能效比的影响。本文提出了三种不同的加法器树设计方案，一种是 28T 和 16T 全加器交替的加法树电路，一种是改进后的 16T 的加法树电路，还有一种是混合华莱士树结构的加法树电路。针对它们各自的优势和劣势进行了详细分析。通过仿真实验，本文对这些加法器树的性能进行了评估，以找出最适合的设计方案。
4. 本文设计了一个基于 SRAM 的可配置逻辑运算存算一体架构，包括存储计算阵列、外围控制电路、加法器树和累加器，并对整个架构进行了仿真分析，与不同设计方案进行性能指标比对。通过数据的对比分析，本设计具有高能效、高吞吐量的优势。

5.2 研究方向展望

本文提出了一种基于 SRAM 的可配置逻辑运算存内计算架构,在经过仿真优化后,展现了其在能效比和吞吐量方面的优势。但仍然存在一些局限性:首先,该设计尚未实际制造,因此缺乏更贴近实际应用的性能评估;其次,由于没有使用测试数据集,无法评估其在神经网络中的实际应用效果;最后,目前本设计仅限于逻辑运算,可以扩展更复杂的计算功能,如复杂的数据分析和处理算法。集成更复杂的数学和逻辑运算功能,如高级统计分析、图像处理算法,甚至是机器学习模型的直接运算,将极大提升 CIM 系统的应用范围和效率。

存内计算作为一种具有巨大潜力的技术,未来将在计算效率、能效、多功能性和系统集成等方面实现重大突破,为处理日益增长的全球数据需求提供强有力的支撑。

参考文献

- [1] 张心怡. 破除“存储墙”, 存内计算来助力[N]. 中国电子报, 2022-08-23(007).
- [2] J. Wang, C. Charles. A 28-nm Compute SRAM With Bit-Serial Logic Arithmetic Operations for Programmable In-Memory Vector Computing[J]. IEEE Journal of Solid-State Circuits, 2020, 55(1):76–87.
- [3] 陈涛.基于 FPGA 的 DDR3 内存控制器的研究与设计[D].成都理工大学,2015.
- [4] V. Sze. Designing Hardware for Machine Learning: The Important Role Played by Circuit Designers[J]. IEEE Solid-State Circuits Magazine, 2017, 9(4):46–54.
- [5] K. Ando, K. Orimo, H. Yonekawat, et al. BRein Memory: A Single-Chip Binary/Ternary Reconfigurable in-Memory Deep Neural Network Accelerator Achieving 1.4 TOPS at 0.6 W[J]. IEEE Journal of Solid-State Circuits, 2018, 53(4):983–994.
- [6] M. Horowitz. Computing’s energy problem[C]. IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 2014:10–14.
- [7] K. William. Cellular Logic-in-Memory Arrays[J]. IEEE Transactions on Computers, 1969, 18(8):719-728.
- [8] Wang J, Wang X, Eckert C, et al. A 28-nm Compute SRAM With Bit-Serial Logic/Arithmetic Operations for Programmable In-Memory Vector Computing[J]. IEEE Journal of Solid-State Circuits, 2020, 55(1): 76-86.
- [9] H. -T. Lue et al. Optimal Design Methods to Transform 3D NAND Flash into a High-Density, High-Bandwidth and Low-Power Nonvolatile Computing in Memory (nvCIM) Accelerator for Deep-Learning Neural Networks (DNN) [C]. 2019 IEEE International Electron Devices Meeting, San Francisco, CA, USA, 2019, pp. 38.1.1-38.1.4.
- [10] S. Xie, C. Ni, A. Sayal, P. Jain, F. Hamzaoglu and J. P. Kulkarni. eDRAM-CIM: Compute-In-Memory Design with Reconfigurable Embedded-Dynamic-Memory Array Realizing Adaptive Data Converters and Charge-Domain Computing[C]. 2021 IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 2021, pp. 248-250.

- [11]Jung, S., Lee, H., Myung, S. et al. A crossbar array of magnetoresistive memory devices for in-memory computing.[J] Nature 601, 211–216 (2022).
- [12]H. Cai et al. A 28nm 2Mb STT-MRAM Computing-in-Memory Macro with a Refined Bit-Cell and 22.4 - 41.5TOPS/W for AI Inference[C]. 2023 IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 2023, p p. 500-502.
- [13]M. Davies et al. Loihi: A Neuromorphic Manycore Processor with On-Chip Learning[C]. IEEE Micro, vol. 38, no. 1, pp. 82-99, January/February 2018.
- [14]X. Si et al. A Twin-8T SRAM Computation-In-Memory Macro for Multiple-Bit CNN-Based Machine Learning[C]. 2019 IEEE International Solid State Circuits Conference, San Francisco, CA, USA, 2019, pp. 396-398.
- [15]J. Yue et al. A 65nm Computing-in-Memory-Based CNN Processor with 2.9-to-35.8TOPS/W System Energy Efficiency Using Dynamic-Sparsity Performance-Scaling Architecture and Energy-Efficient Inter/Intra-Macro Data Reuse[C]. 2020 IEEE International Solid State Circuits Conference, San Francisco, CA, USA, 2020, pp. 234-236.
- [16]J. -W. Su et al. Two-Way Transpose Multibit 6T SRAM Computing-in-Memory Macro for Inference-Training AI Edge Chips[J]. IEEE Journal of Solid-State Circuits, vol. 57, no. 2, pp. 609-624, Feb. 2022.
- [17]J. -W. Su et al. A 28nm 384kb 6T-SRAM Computation-in-Memory Macro with 8b Precision for AI Edge Chips[C]. 2021 IEEE International Solid State Circuits Conference (ISSCC), San Francisco, CA, USA, 2021, pp. 250-252.
- [18]P. -C. Wu et al. A 22nm 832Kb Hybrid-Domain Floating-Point SRAM In-Memory-Compute Macro with 16.2-70.2TFLOPS/W for High-Accuracy AI-Edge Devices[C]. 2023 IEEE International Solid- State Circuits Conference, San Francisco, CA, USA, 2023, pp. 126-128, doi: 10.1109/ISSCC42615.2023.10067527.
- [19]B. Yan et al. A 1.041-Mb/mm² 27.38-TOPS/W Signed-INT8 Dynamic-Logic-Based ADC-less SRAM Compute-in-Memory Macro in 28nm with Reconfigurable Bitwise Operation for AI and Embedded Applications[C]. 2022 IEEE International Solid State Circuits Conference, San Francisco, CA, USA, 2022, p p. 188-190.

- [20] A. Sridharan, S. Angizi, S. K. Cherupally, F. Zhang, J. -S. Seo and D. Fan. A 1.23-GHz 16-kb Programmable and Generic Processing-in-SRAM Accelerator in 65nm[C]. ESSCIRC 2022 IEEE 48th European Solid State Circuits Conference, Milan, Italy, 2022, pp. 153-156.
- [21] S. Liu et al. A 28nm 53.8TOPS/W 8b Sparse Transformer Accelerator with In-Memory Butterfly Zero Skipper for Unstructured-Pruned NN and CIM-Based Local-Attention-Reusable Engine[C]. 2023 IEEE International Solid State Circuits Conference, San Francisco, CA, USA, 2023, pp. 250-252.
- [22] H. Fujiwara et al. A 7nm 2.1GHz Dual-Port SRAM with WL-RC Optimization and Dummy-Read-Recovery Circuitry to Mitigate Read-Disturb-Write Issue[C]. 2019 IEEE International Solid State Circuits Conference, San Francisco, CA, USA, 2019, pp. 390-392.
- [23] Y. -D. Chih et al. An 89TOPS/W and 16.3TOPS/mm² All-Digital SRAM-Based Full-Precision Compute-In Memory Macro in 22nm for Machine-Learning Edge Applications[C]. 2021 IEEE International Solid State Circuits Conference (ISSCC), San Francisco, CA, USA, 2021, pp. 252-254.
- [24] H. Mori et al. A 4nm 6163-TOPS/W/b 4790-TOPS/mm² SRAM Based Digital-Computing-in-Memory Macro Supporting Bit-Width Flexibility and Simultaneous MAC and Weight Update[C]. 2023 IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 2023, pp. 132-134.
- [25] Kumar V., Tomar V. K. A comparative performance analysis of 6T, 7T and 8T SRAM cells in 18nm FinFET technology[A]. 2020 International Conference on Power Electronics & IoT Applications in Renewable Energy and its Control (PARC)[C]. 2020.
- [26] 司鑫. 面向人工智能的嵌入式存储器及存内计算电路设计[D]. 电子科技大学, 2021.
- [27] Muscedere R., Leboeuf K. A dynamic address decode circuit for implementing range addressable look-up tables[A]. IEEE International Symposium on Circuits & Systems[C]. IEEE, 2008.
- [28] Sun, Z., Kvatinsky, S., Si, X. et al. A full spectrum of computing-in-memory technologies[J]. Nat Electron 6, 823–835 (2023).
- [29] C. F. Lee. A 12nm 121-TOPS/W 41.6-TOPS/mm² All Digital Full Precision SRAM-based Compute-in-Memory with Configurable Bit-width For AI Edge

- Applications[C]. 2022 IEEE Symposium on VLSI Technology and Circuits, Honolulu, HI, USA, 2022:24-25.
- [30]M. Chang, S. Spetalnick, B. Crafton, et al. A 40nm 60.64TOPS/W ECC-Capable Compute-in-Memory/Digital 2.25MB/768KB RRAM/SRAM System with Embedded Cortex M3 Microprocessor for Edge Recommendation Systems[C]. 2022 IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 2022:1-3.
- [31]E. Lee, G. Shin, S. Jeong. A charge-domain scalable-weight in-memory computing macro with dual-sram architecture for precision-scalable dnn accelerators[J]. IEEE Transactions on Circuits and Systems I: Regular Papers, 2021, 58(8):3305–3316.
- [32]M. E. Sinangil, B. Erbagci, R. Naous, et al. A 7-nm Compute-in-Memory SRAM Macro Supporting Multi-Bit Input, Weight and Output and Achieving 351 TOPS/W and 372.4 GOPS[J]. IEEE Journal of Solid-State Circuits, 2021, 56(1):188–198.
- [33]A. Jaiswal, I. Chakraborty, A. Agrawal. 8T SRAM Cell as a Multibit Dot-Product Engine for beyond Von Neumann Computing[J]. IEEE Transactions on Very Large-Scale Integration (VLSI) Systems, 2019, 27(11):2556–2567.
- [34]Q. Dong, M. Sinangil, B. Erbagci, et al. A 351TOPS/W and 372.4GOPS Compute-in-Memory SRAM Macro in 7nm FinFET CMOS for Machine-Learning Applications[C]. IEEE International Solid-State Circuits Conference, San Francisco, CA, USA, 2020:242-243.
- [35]Seevinck E, List F J, Lohstroh J. Static-noise margin analysis of MOS SRAM cells[J]. IEEE Journal of Solid-State Circuits, 1987, 22(5): 748-754.
- [36]Qiu H, Mizutani T, Saraya T, et al. Comparison and statistical analysis of four write stability metrics in bulk CMOS static random access memory cells[J]. JAPANESE JOURNAL OF APPLIED PHYSICS, 2015, 54(4).
- [37]N. Weste and K. Eshraghian, Principles of CMOS VLSI Design—A Systems Perspective[M], Addison-Wesley, 1993
- [38]M. M. Vai, VLSI Design[M]. Boca Raton, FL: CRC, 2001.
- [39]张振伟. 应用于神经网络的存内计算电路设计[D].电子科技大学,2023.
- [40]王梓霖. 面向低功耗嵌入式系统的存内计算电路设计[D].电子科技大学,2023.

- [41]M. Alioto and G. Palumbo. Analysis and comparison on full adder block in submicron technology[J]. IEEE Trans. Very Large Scale (VLSI) Syst., vol. 10, no. 6, pp. 806–823, Dec. 2002.
- [42]A. P. Chandrakasan and R. W. Brodersen, Low Power Digital CMOS Design [M]. Norwell, MA: Kluwer, 1995.
- [43]P. J. Song and G. De Micheli. Circuit and architecture trade-offs for high-speed multiplication[J]. IEEE J. Solid-State Circuits, vol. 26, no. 9, pp. 1184–1198, Sep. 1991.
- [44]Z. Wang, G. Jullien, and W. C. Miller. A new design technique for column compression multipliers[J]. IEEE Trans. Comput., vol. 44, no. 8, pp. 962–970, Aug. 1995.
- [45]王志南. 基于 0.18um 的加法器芯片研究与设计[D].山东大学,2020.
- [46]Mattausch H. J., Gyohten T., Soda Y., et al. Compact associative-memory architecture with fully parallel search capability for the minimum Hamming distance[J]. IEEE Journal of Solid-State Circuits, 2002, V37(2):218-227.
- [47]Oike Y., Ikeda M., Asada K. A high-speed and low-voltage associative co-processor with exact Hamming/Manhattan-distance estimation using word-parallel and hierarchical search architecture[J]. IEEE Journal of Solid-State Circuits, 2004, V39(8):1383-1387.
- [48]H. Kim et al., “A 1-16b Precision Reconfigurable Digital In-Memory Computing Macro Featuring Column-MAC Architecture and Bit-Serial Computation,” ESSCIRC[C] , 2019, pp. 345-348.